SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 기본

Queue

Python

학습시작

번호표를 뽑은 순서대로 부르는 은행 창구!

은행에서는 고객이 번호표를 뽑고 기다립니다. 번호표는 먼저 온 사람부터 부르기 때문에 아래와 같이 번호표를 뽑았다고 가정해 봅시다.

1번 고객 → 2번 고객 → 3번 고객

이후 창구 직원이 두 명의 고객을 호출했다면, 호출된 고객 번호는 어떤 순서일까요?

- 큐는 먼저 들어간 데이터가 먼저 나오는 구조입니다. (First-In-First-Out)
- 실생활 예시(은행 창구 번호표)를 통해 큐의 동작 원리를 쉽게 이해할 수 있어요.
- 이후에는 파이썬 코드로 append()와 pop(0)을 써서 구현해 볼 수 있어요!

학습목표

- 큐 자료구조의 개념과 기본 연산을 설명할 수 있다.
- 선형 큐와 원형 큐에 대해 설명하고 이를 구현할 수 있다.
- 연결 큐에 대해 설명할 수 있다.
- 우선순위 큐의 개념과 알고리즘을 설명할 수 있다.
- 큐의 활용인 버퍼에 대해 설명할 수 있다.
- 큐를 활용한 너비 우선 탐색 알고리즘에 대하여 설명할 수 있다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

큐(Queue)

- 큐
- 선형 큐
- 원형 큐
- 연결 큐
- 우선순위 큐

큐의 활용

- 버퍼
- BFS

큐(Queue)

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

큐

큐(Queue)의 개념

선형 자료구조 :
자료 간의 관계가 1:1의 관계를 갖는다. (예: 배열, 리스트)

<개념1_SW문제해결 기본_Queue_선형구조>

큐
Queue

먼저 들어온 데이터가 먼저 나가는 선형 자료구조

스택과 마찬가지로 삽입과 삭제의 위치가 제한적인 자료구조로
큐의 뒤에서는 삽입만 하고, 앞에서는 삭제만 이루어지는 구조입니다.

큐(Queue)의 구조

선입선출

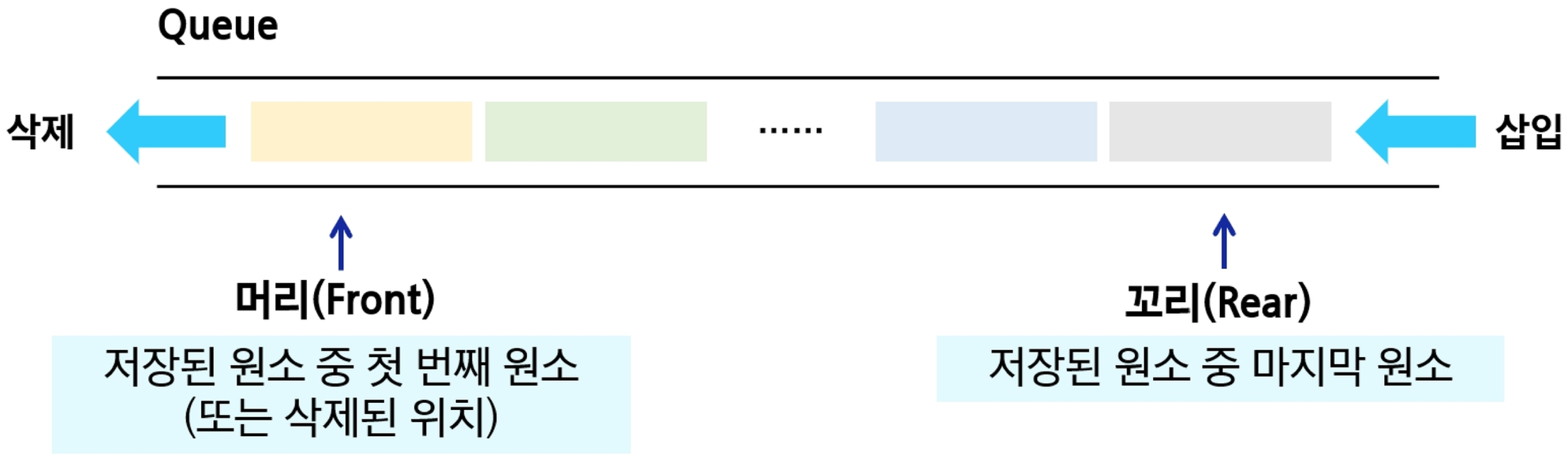
FIFO, First In First Out

가장 먼저 넣은 자료가 가장 먼저 나오는 것

예를 들어 큐에 1, 2, 3을 차례대로 넣으면 꺼낼 때도 1, 2, 3이 순서대로 나오게 됩니다.

큐의 또 다른 예로는 서비스 대기행렬이 있습니다.

큐(Queue)의 구조



<그림1_SW문제해결 기본_Queue_큐의 구조 예시>

큐의 기본 연산

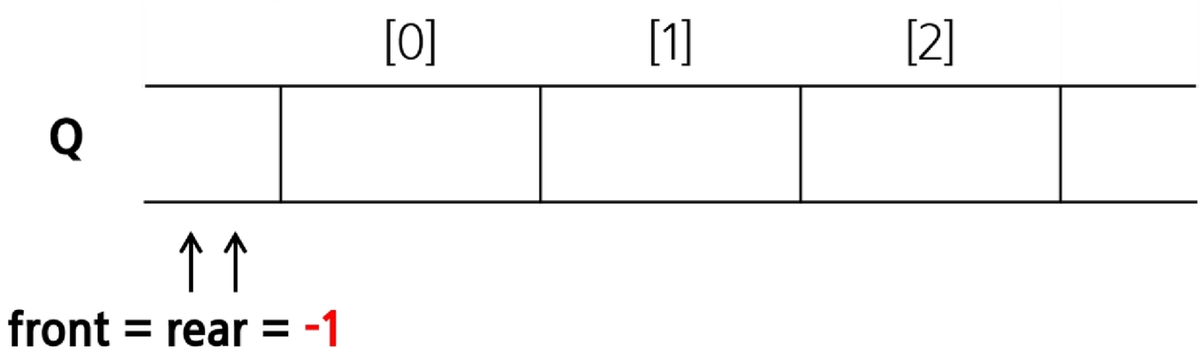
- 삽입 : enqueue
- 삭제 : dequeue

연산	기능
enqueue(item)	큐의 뒤쪽(rear 다음)에 원소를 삽입하는 연산
dequeue()	큐의 앞쪽(front)에서 원소를 삭제하고 반환하는 연산
create_queue()	공백 상태의 큐를 생성하는 연산
is_empty()	큐가 공백상태인지를 확인하는 연산
is_full()	큐가 포화상태인지를 확인하는 연산
qpeek()	큐의 앞쪽(front)에서 원소를 삭제 없이 반환하는 연산

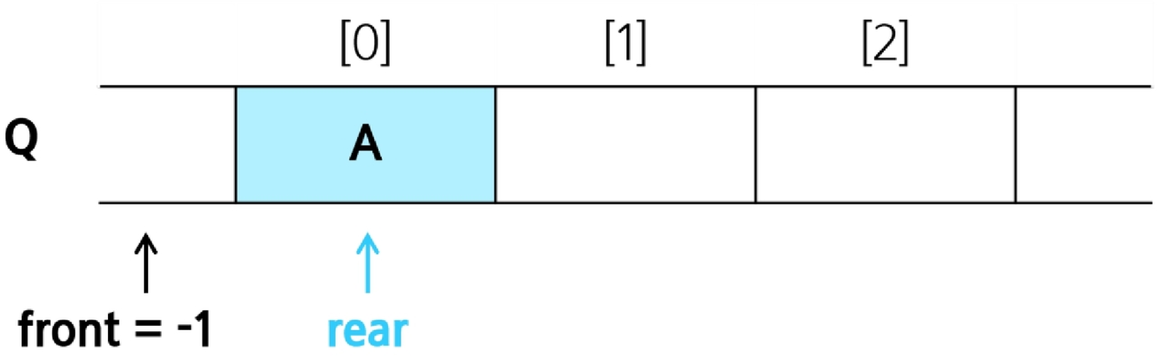
<표1_SW문제해결 기본_Queue_큐의 기본 연산>

큐의 연산 과정

1단계 공백 큐 생성 : create_queue()



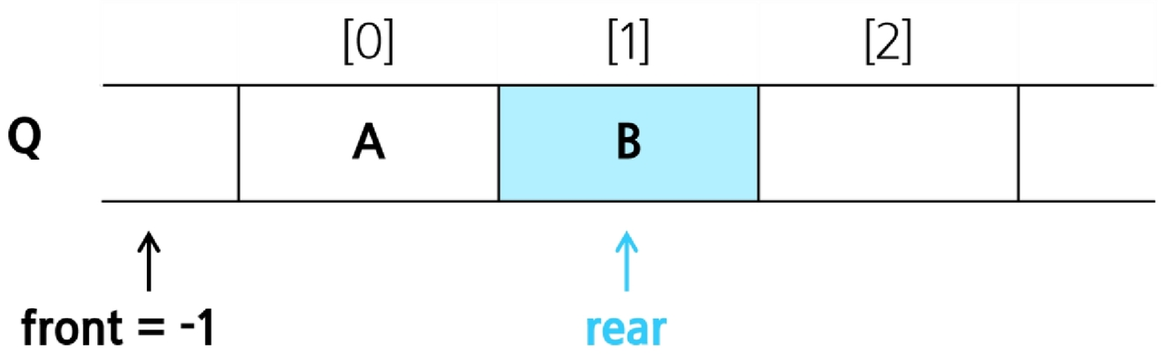
2단계 원소 A 삽입 : enqueue(A)



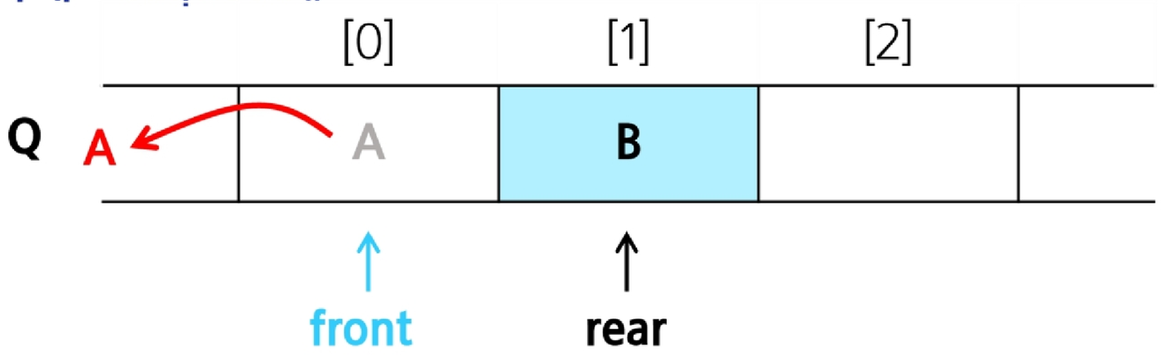
<그림2_SW문제해결 기본_Queue_큐의 생성과 원소 A 삽입>

큐의 연산 과정

3단계 원소 B 삽입 : enqueue(B)



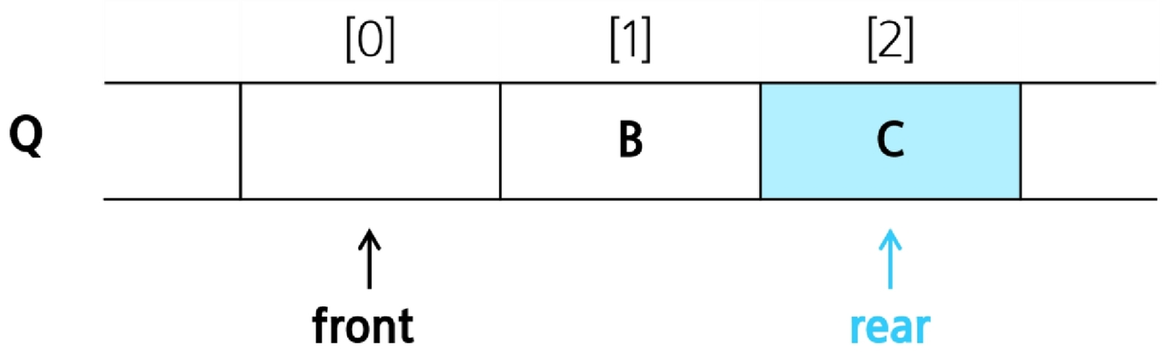
4단계 원소 반환/삭제 : dequeue()



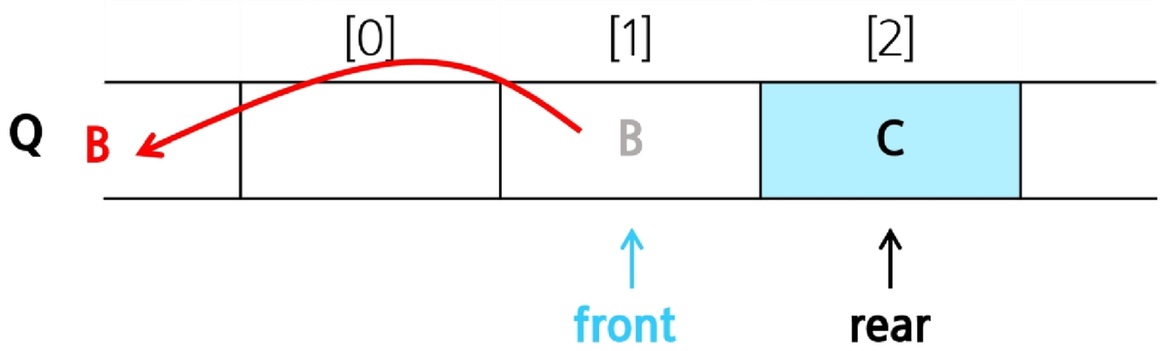
<그림3_SW문제해결 기본_Queue_큐에 B 삽입 및 삭제>

큐의 연산 과정

5단계 원소 C 삽입 : enqueue(C);



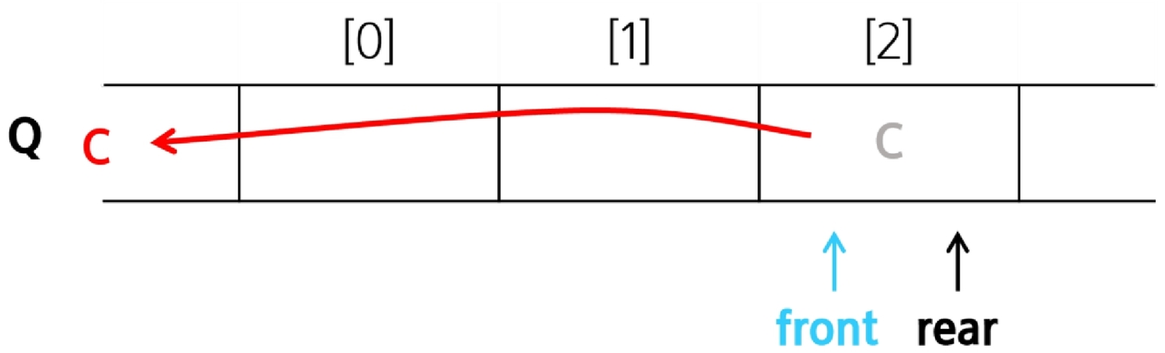
6단계 원소 반환/삭제 : dequeue();



<그림4_SW문제해결 기본_Queue_큐에 C 삽입 및 삭제>

큐의 연산 과정

7단계 원소 반환/삭제 : dequeue();



<그림5_SW문제해결 기본_Queue_큐의 삭제>

선형 큐

선형 큐의 개념

선형 큐
Linear Queue

데이터를 일렬로 저장하며,
앞에서 꺼내고 뒤에 넣는 기본 큐 구조

선형 큐

- 구현
  - 배열이나 연결 리스트로 구현할 수 있습니다.
  - 배열로 구현한 경우 큐의 크기는 배열의 크기와 같습니다.
  - front: 가장 최근에 삭제된 원소의 인덱스입니다.
  - rear: 마지막으로 저장된 원소의 인덱스입니다.

- 상태 표현
  - 초기 상태 : front = rear = -1
  - 공백 상태 : front == rear
  - 포화 상태 : rear == n-1 (n : 배열의 크기, n-1 : 배열의 마지막 인덱스)

선형 큐의 구현

① 초기 공백 큐 생성 : create_queue()

- 크기 n인 1차원 배열 생성
- front와 rear를 -1로 초기화

```
# create_queue()에 해당
q = [0] * n
front = -1
rear = -1
```

선형 큐의 구현

② 삽입 : enqueue(item)

- 마지막 원소 뒤에 새로운 원소를 삽입하기 위해

1) rear 값을 하나 증가시켜 새로운 원소를 삽입할 자리를 지정

2) 그 인덱스에 해당하는 배열 원소 q[rear]에 item을 저장

```
def enqueue(item) :
        global rear
        if is_full() :
                print("Queue_Full")
        else :
                rear = rear + 1
                q[rear] = item
```

선형 큐의 구현

③ 삭제 : dequeue()

- 가장 앞에 있는 원소를 삭제하기 위해

1) front 값을 하나 증가시켜 큐에 남아 있는 첫 번째 원소로 이동

2) 첫 번째 원소를 반환함으로써 삭제와 동일한 기능을 함

```
def dequeue() :
        global front
        if is_empty() :
                print("Queue_Empty")
        else :
                front = front + 1
                return q[front]
```

선형 큐의 구현

④ 공백 상태 및 포화 상태 검사 : is_empty(), is_full()

- 공백 상태 : front == rear
- 포화 상태 : rear == n-1 (n : 배열의 크기, n-1 : 배열의 마지막 인덱스)

```
def is_empty() :
        return front == rear

def is_full() :
        return rear == len(q) - 1
```

선형 큐의 구현

⑤ 검색 : qpeek()

- 가장 앞에 있는 원소를 검색하여 반환하는 연산
- 현재 front의 한 자리 뒤(front+1)에 있는 원소, 즉 큐의 첫 번째에 있는 원소를 반환

```
def qpeek() :
        if is_empty() :
                print("Queue_Empty")
        else :
                return q[front+1]
```

선형 큐 연습 문제

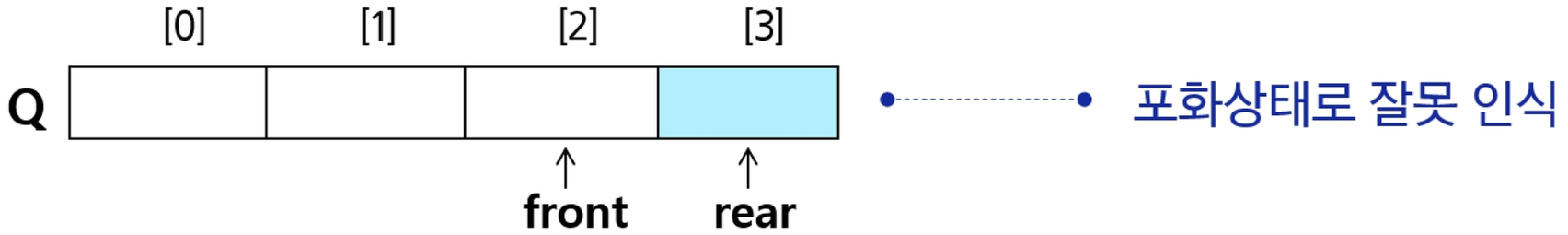
- 큐를 구현하여 다음 동작을 확인하기

- 세 개의 데이터 1, 2, 3을 차례로 큐에 삽입하고
- 큐에서 세 개의 데이터를 차례로 꺼내서 출력한다.
- 1, 2, 3이 출력되어야 함.

원형 큐

선형 큐 사용시의 문제점

- 잘못된 포화상태 인식

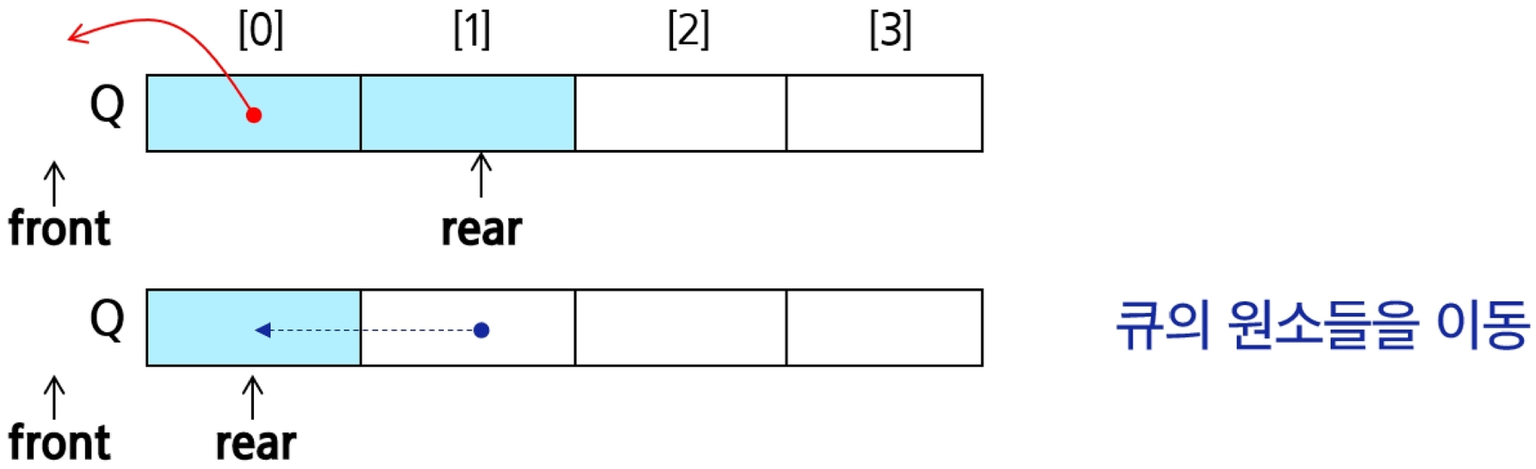


<그림6_SW문제해결 기본_Queue_선형 큐의 잘못된 포화상태 인식>

선형 큐를 이용하여 원소의 삽입과 삭제를 계속할 경우,
배열의 앞부분에 활용할 수 있는 공간이 있음에도 불구하고,
rear==n-1 인 상태 즉, 포화상태로 인식하여 더 이상의 삽입을 수행하지 않게 됩니다.

해결 방법 1

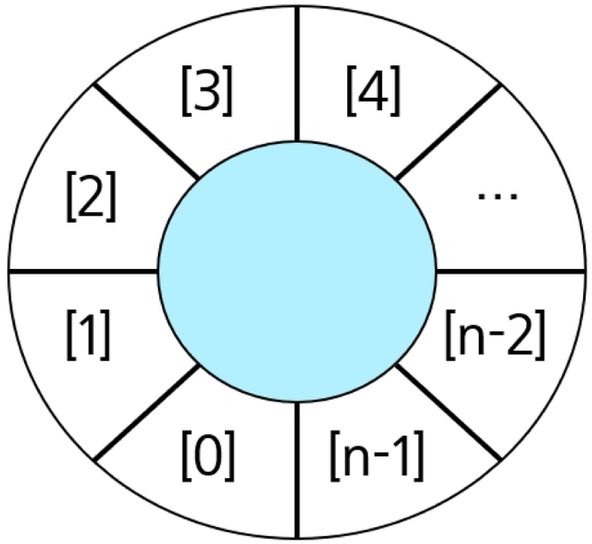
- 매 연산이 이루어질 때마다 저장된 원소들을 배열의 앞부분으로 모두 이동
  - 그러나, 이 방법은 원소 이동에 많은 시간이 소요되어 큐의 효율성이 급격히 떨어집니다.



<그림7_SW문제해결 기본_Queue_dequeue 후 원소 이동>

해결 방법 2

- 1차원 배열을 사용
  - 논리적으로는 배열의 처음과 끝이 연결되어 원형 형태를 이룬다고 가정하고 사용합니다.



원형 큐의 논리적 구조

<그림8_SW문제해결 기본_Queue_원형 큐의 논리적 구조>

원형 큐의 개념

원형 큐
Circular Queue

선형 큐의 공간 낭비를 막기 위해
처음과 끝이 연결된 구조

원형 큐의 구조

- 초기 공백 상태
  - front = rear = 0

- Index의 순환
  - front 또는 rear의 위치가 배열의 마지막 인덱스인 n-1을 가리킵니다.
  - 그 다음에는 논리적 순환을 이루어 배열의 처음 인덱스인 0으로 이동해야 합니다.
  - 이를 위해 나머지 연산자(%: mod)를 사용합니다.

- front 변수
  - 공백 상태와 포화 상태 구분을 쉽게 하기 위해 front가 있는 자리는 사용하지 않고 항상 빈 자리로 둡니다.

선형 큐와 원형 큐 비교

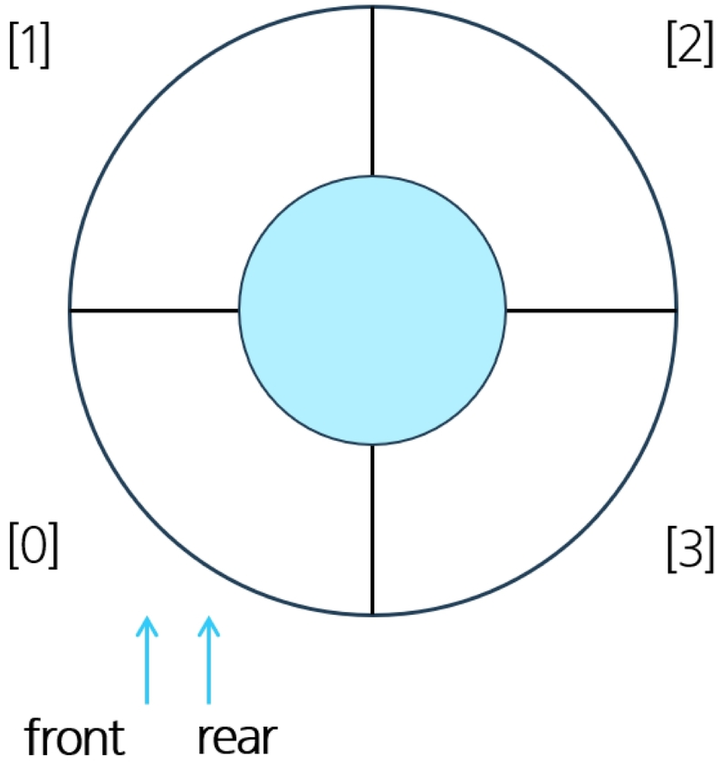
- 삽입 위치 및 삭제 위치

	삽입 위치	삭제 위치
선형 큐	rear = rear + 1	front = front + 1
원형 큐	rear = (rear + 1) mod n	front = (front + 1) mod n

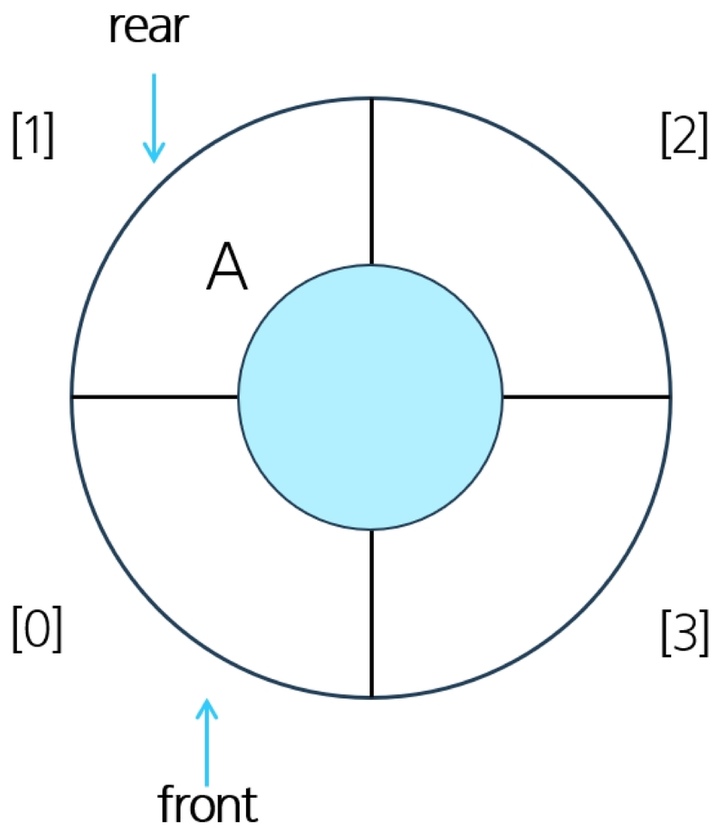
<표2_SW문제해결 기본_Queue_선형 큐와 원형 큐의 비교>

원형 큐의 연산 과정

1단계 create Queue



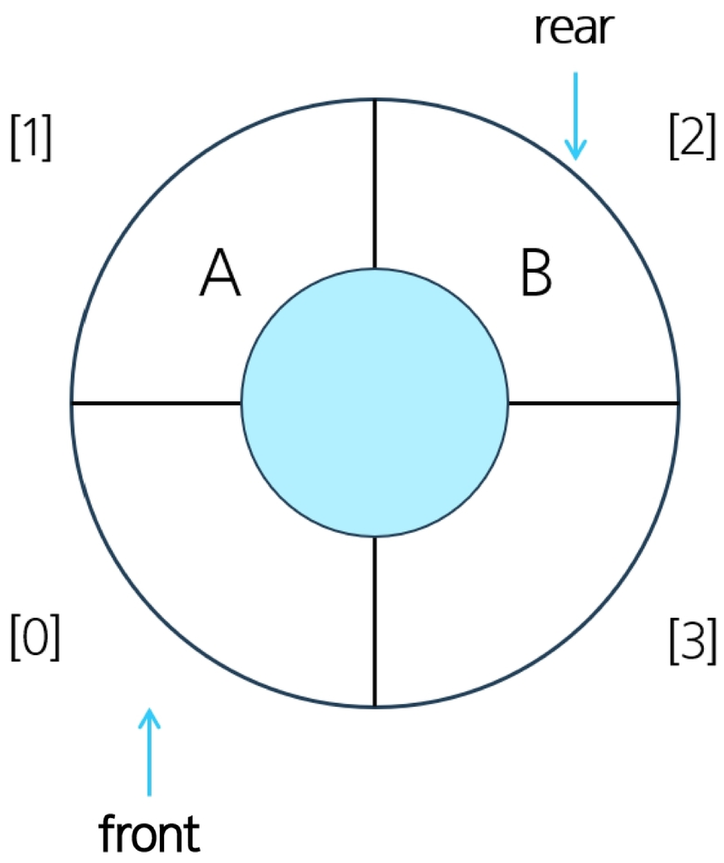
2단계 enqueue(A);



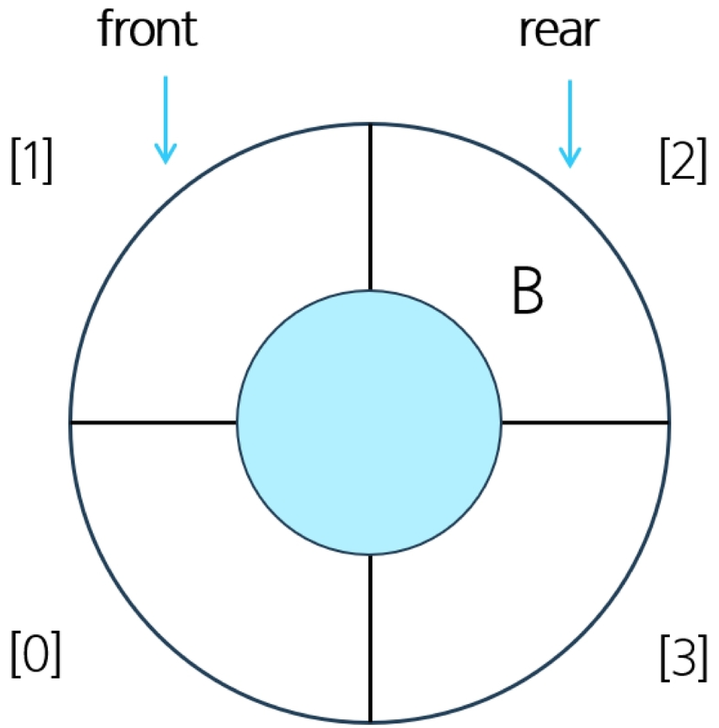
<그림9_SW문제해결 기본_Queue_원형 큐의 연산 1>

원형 큐의 연산 과정

3단계 enqueue(B);



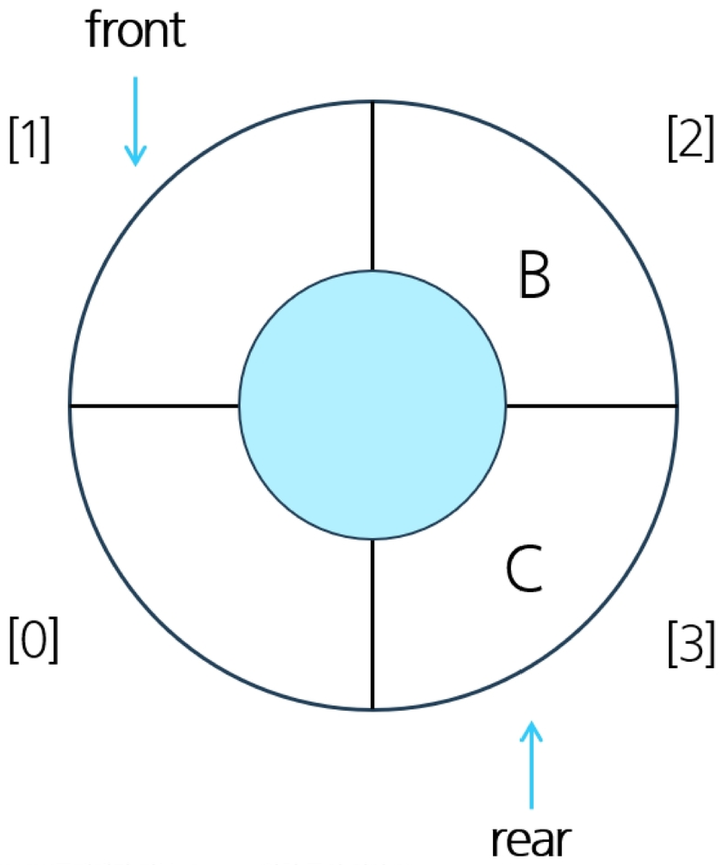
4단계 dequeue();



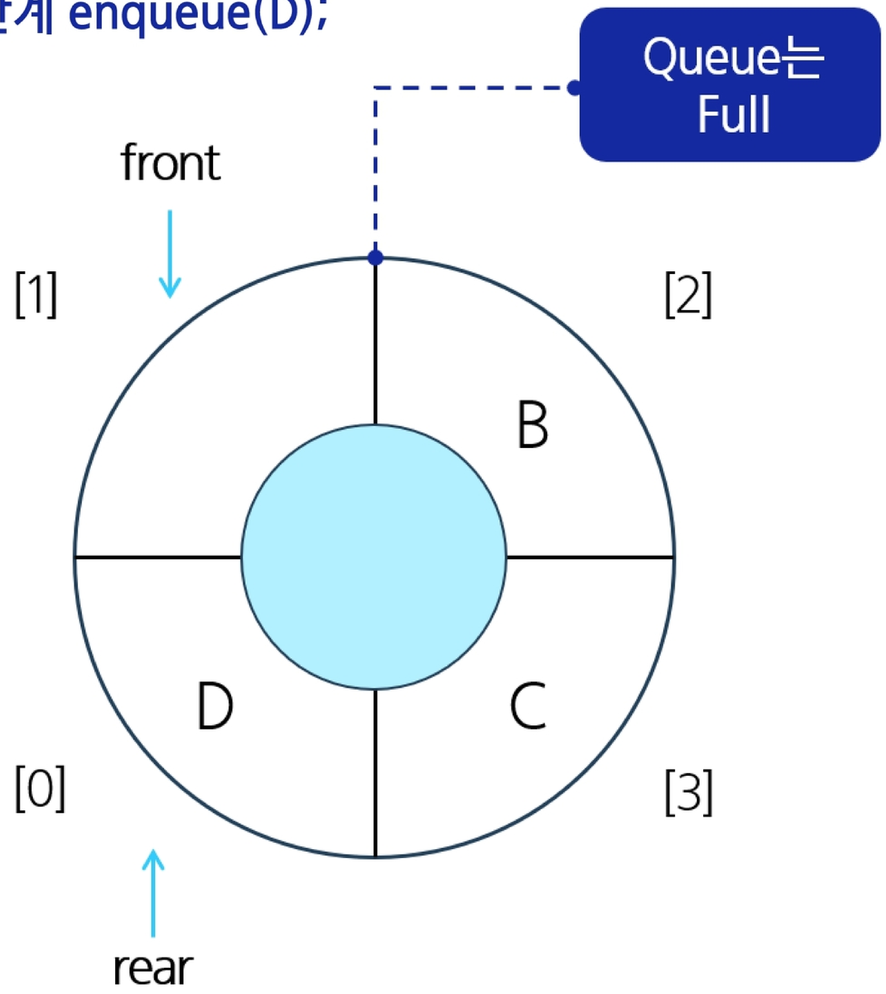
<그림10_SW문제해결 기본_Queue_원형 큐의 연산 2>

원형 큐의 연산 과정

5단계 enqueue(C);



6단계 enqueue(D);



<그림11_SW문제해결 기본_Queue_원형 큐의 연산 3>

원형 큐의 구현

① 초기 공백 큐 생성

- 크기 n인 1차원 배열 생성
- front와 rear를 0으로 초기화

```
cq = [0] * n
front = rear = 0
```

원형 큐의 구현

② 삽입 : enqueue(item)

- 마지막 원소 뒤에 새로운 원소를 삽입하기 위해

1) rear 값을 조정하여 새로운 원소를 삽입할 자리를 지정

rear <- (rear + 1) mod n

2) 그 인덱스에 해당하는 배열 원소 cq[rear]에 item을 저장

```
def enqueue(item) :
    global rear
    if is_full() :
        print("Queue_Full")
    else :
        rear = (rear + 1) % len(cq)
        cq[rear] = item
```

원형 큐의 구현

③ 삭제 : dequeue()

- 가장 앞에 있는 원소를 삭제하기 위해

1) front 값을 조정하여 삭제할 자리를 지정

front <- (front + 1) mod n

2) 새로운 front 원소를 반환함으로써 삭제와 동일한 기능을 함

```
def dequeue() :
    global front
    if is_empty() :
        print("Queue_Empty")
    else :
        front = (front + 1) % len(cq)
        return cq[front]
```

원형 큐의 구현

④ 공백상태 및 포화상태 검사 : is_empty(), is_full()

- 공백상태 : front == rear
- 포화상태 : 삽입할 rear의 다음 위치 == 현재 front

  (rear + 1) mod n == front

```
def is_empty() :
        return front == rear

def is_full() :
        return (rear+1) % len(cq) == front
```

연결 큐

연결 큐의 개념

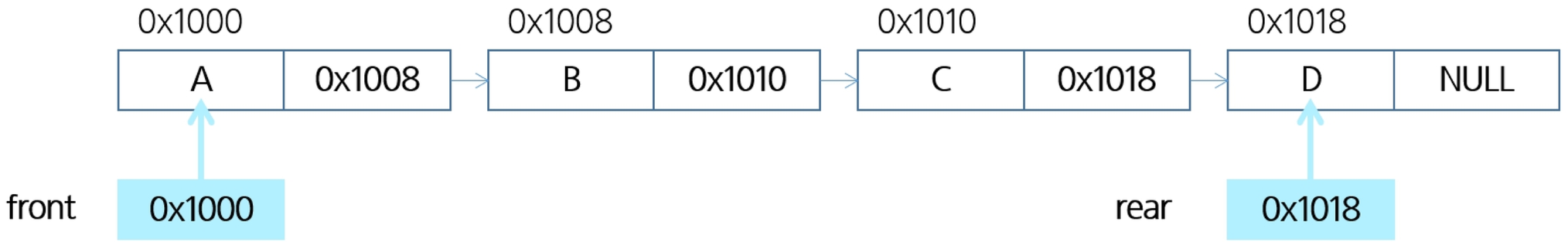
연결 큐
Linked Queue

연결 리스트를 이용해 구현한 큐

연결 큐의 구조

- 단순 연결 리스트(Linked List)를 이용한 큐
  - 큐의 원소 : 단순 연결 리스트의 노드
  - 큐의 원소 순서 : 노드의 연결 순서, 링크로 연결되어 있음
  - front : 첫 번째 노드를 가리키는 링크
  - rear : 마지막 노드를 가리키는 링크

- 상태 표현
  - 초기 상태 : front = rear = NULL
  - 공백 상태 : front = rear = NULL

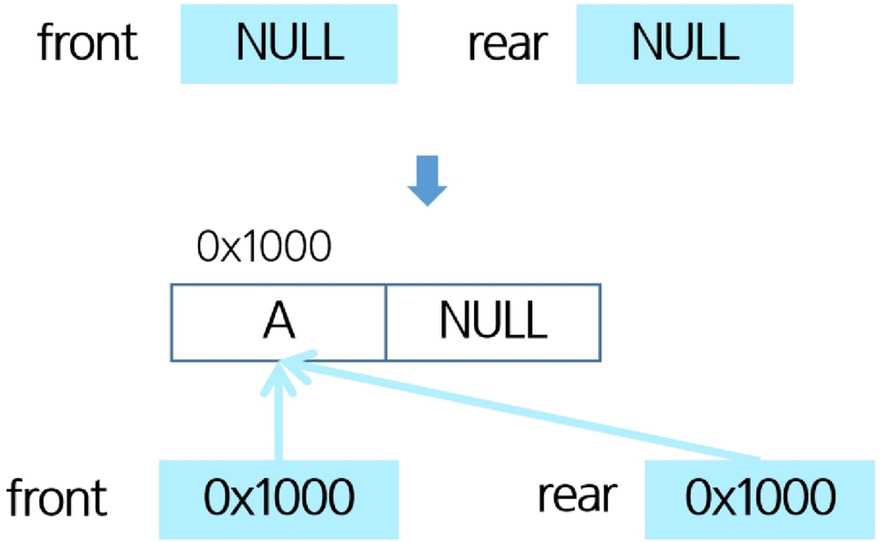


<그림12_SW문제해결 기본_Queue_연결 큐의 구조>

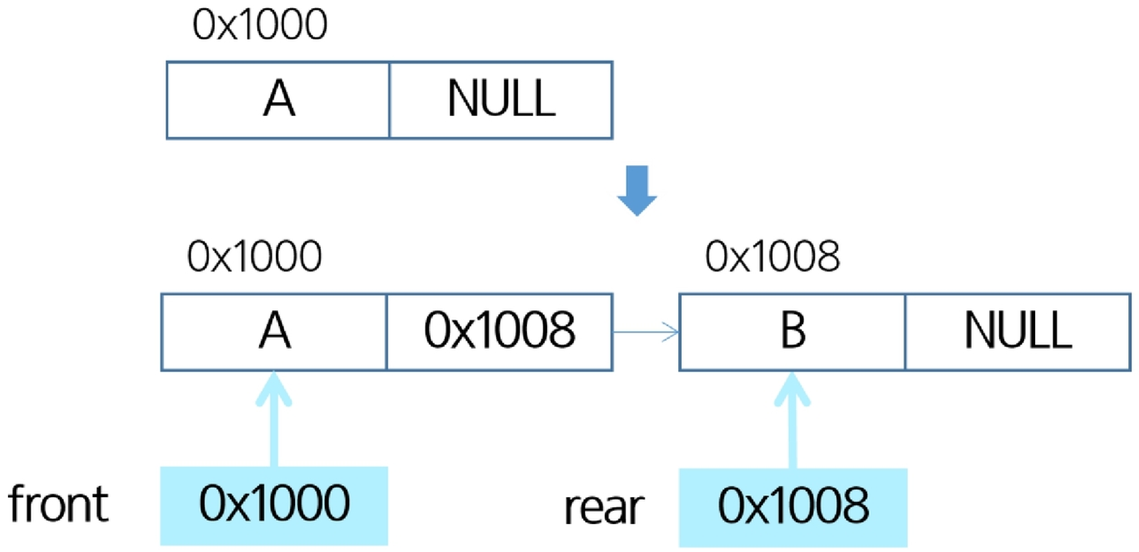
연결 큐의 연산 과정

1단계 공백 큐 생성 : createLinkedQueue();

2단계 원소 A 삽입 : enqueue(A);



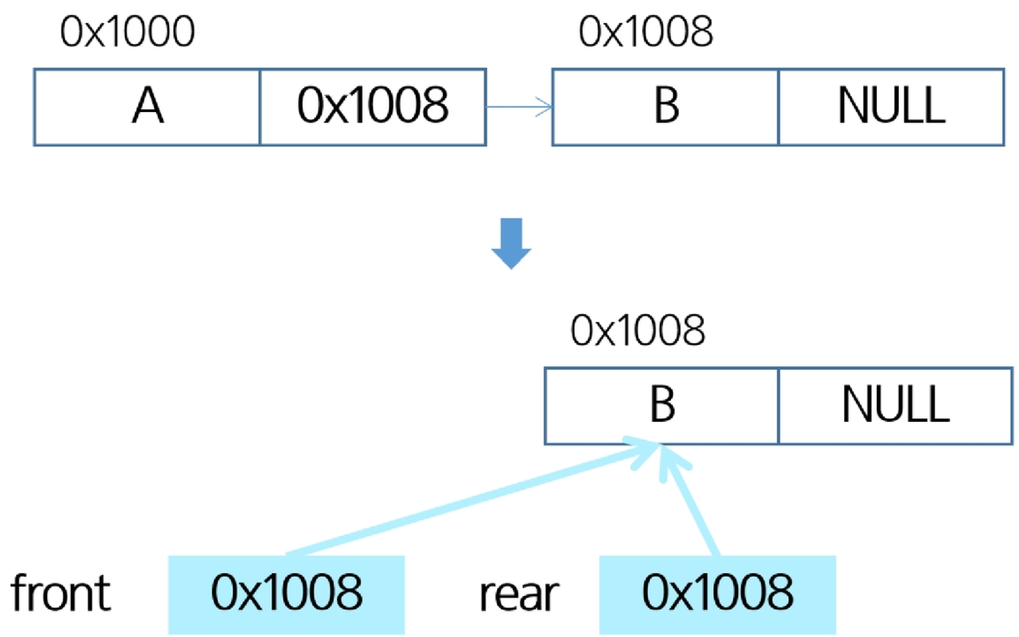
3단계 원소 B 삽입 : enqueue(B);



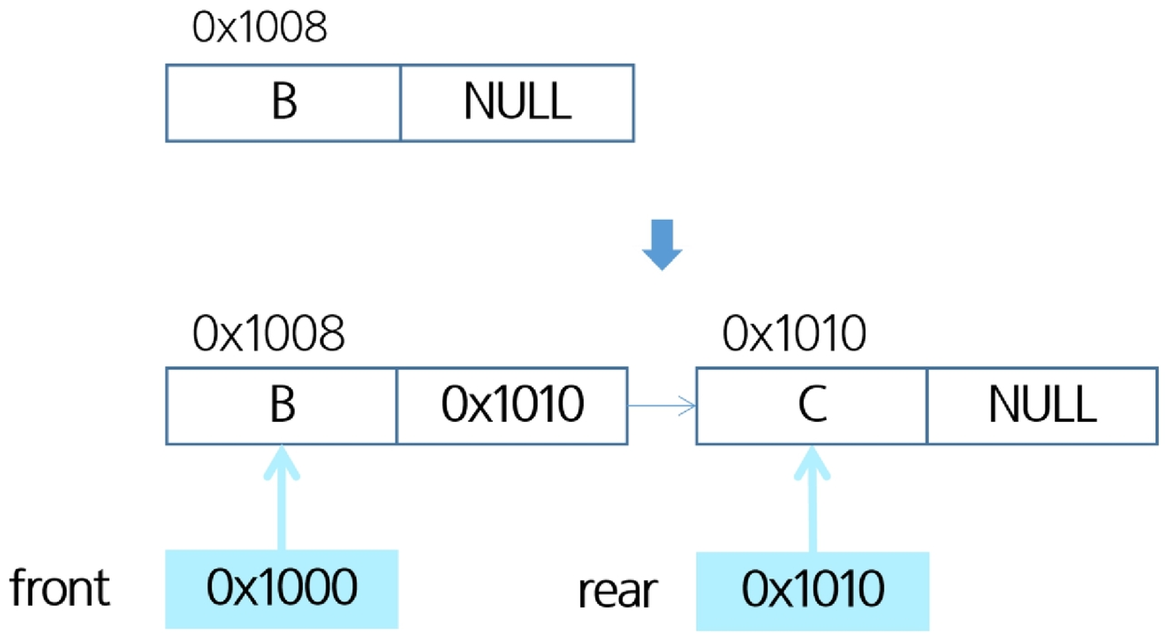
<그림13_SW문제해결 기본_Queue_연결 큐의 삽입 연산>

연결 큐의 연산 과정

4단계 원소 삭제 : dequeue();



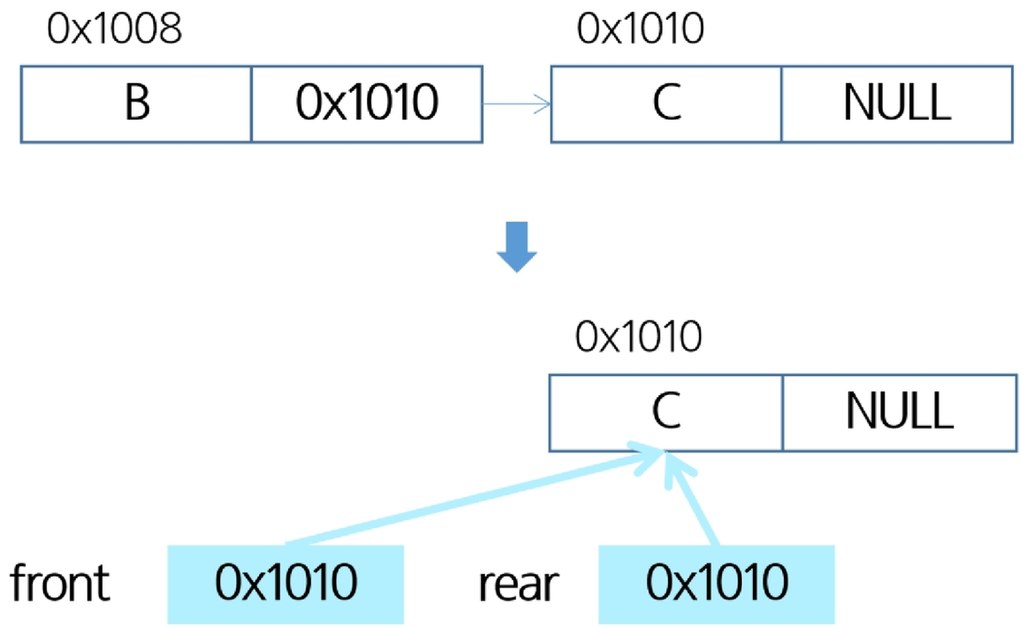
5단계 원소 C 삽입 : enqueue(C);



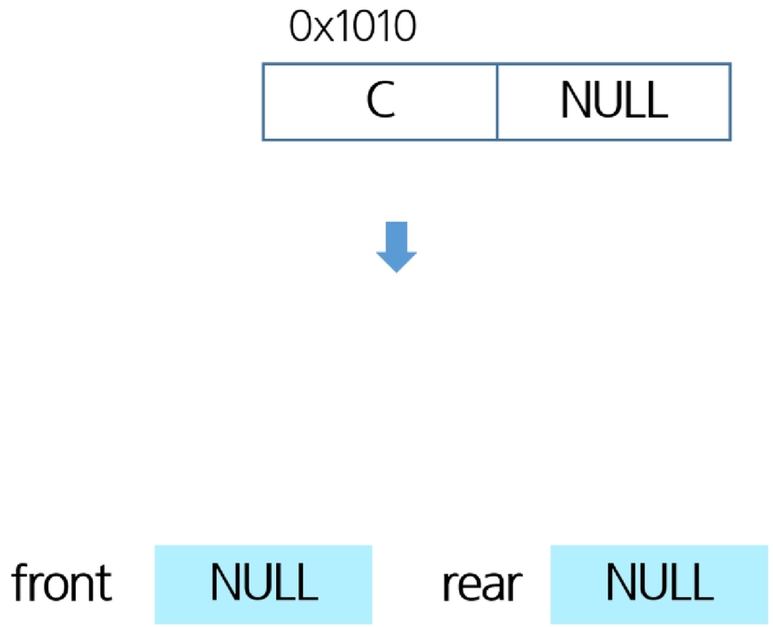
<그림14_SW문제해결 기본_Queue_연결 큐의 삭제와 삽입 연산>

연결 큐의 연산 과정

6단계 원소 삭제 : dequeue();



7단계 원소 삭제 : dequeue();



<그림15_SW문제해결 기본_Queue_연결 큐의 삭제 연산>

deque(덱)

- 컨테이너 자료형 중 하나로 양쪽 끝에서 빠르게 추가와 삭제를 할 수 있는 리스트류 컨테이너

 - 연결 리스트를 직접 만들지 않아도 됩니다.

deque의 연산

- append(x) : 오른쪽에 x 추가
- popleft() : 왼쪽에서 요소를 제거하고 반환, 요소가 없으면 IndexError

```
from collections import deque

q = deque()
q.append(1)     # enqueue()
t = q.popleft() # dequeue()
```

연결 큐의 구현 예(Python 코드)

```
class Node :
    def __init__(self, item, n=None):
        self.item = item
        self.next = n

def enqueue(item) : # 연결 큐의 삽입 연산
    global front, rear
    newNode = Node(item) # 새로운 노드 생성
    if front == None : # 큐가 비어있다면
        front = newNode
    else :
        rear.next = newNode
    rear = newNode
```

연결 큐의 구현 예(Python 코드)

```
def is_empty() :
    return front == None

def dequeue() : # 연결 큐의 삭제 연산
    global front, rear
    if is_empty() :
        print("Queue_Empty")
        return None

    item = front.item
    front = front.next
    if front == None :
        rear = None
    return item
```

연결 큐의 구현 예(Python 코드)

```
def qpeek() :
    return front.item

def print_q() :
    f = front
    s = ""
    while f :
        s += f.item + " "
        f = f.next
    return s

front = None
rear = None
```

우선순위 큐

우선순위 큐의 개념

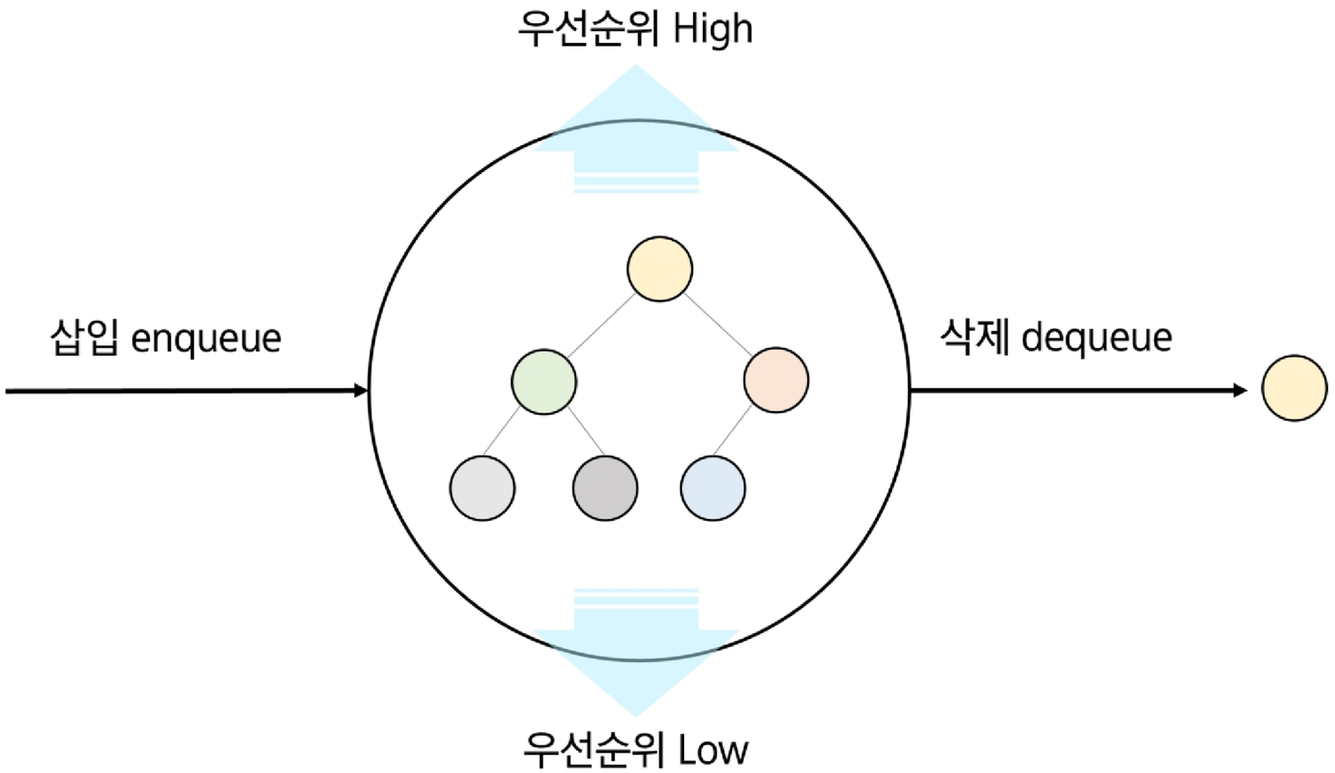
우선순위 큐
Priority Queue

우선순위를 가진 항목들을 저장하는 큐

FIFO 순서가 아니라 우선순위가 높은 순서대로 먼저 나가게 됩니다.
우선순위 큐의 적용 분야로 시뮬레이션 시스템, 네트워크 트래픽 제어, 운영체제의 태스크 스케줄링이 있습니다.

우선순위 큐의 연산

- 삽입 : enqueue
- 삭제 : dequeue



<그림16_SW문제해결 기본_Queue_우선순위 큐의 연산>

배열을 이용한 우선순위 큐

- 배열을 이용하여 자료를 저장
- 원소를 삽입하는 과정에서 우선순위를 비교하여 적절한 위치에 삽입하는 구조
- 가장 앞에 최고 우선순위의 원소가 위치함
- 문제점

- 배열을 사용하므로, 삽입이나 삭제 연산이 일어날 때 원소의 재배치가 발생합니다.
- 이에 소요되는 시간이나 메모리 낭비가 큽니다.

효율적인 우선순위 큐

- 트리 구조인 힙(Heap) 이용
- 트리 단원에서 기본 원리를 다룸

큐의 활용

버퍼

버퍼(Buffer)의 개념

버퍼링 :
버퍼를 활용하는 방식 또는
버퍼를 채우는 동작을 의미한다.

<개념2_SW문제해결 기본_Queue_버퍼링>

버퍼
Buffer

데이터를 한 곳에서 다른 한 곳으로 전송하는 동안
일시적으로 그 데이터를 보관하는 메모리의 영역

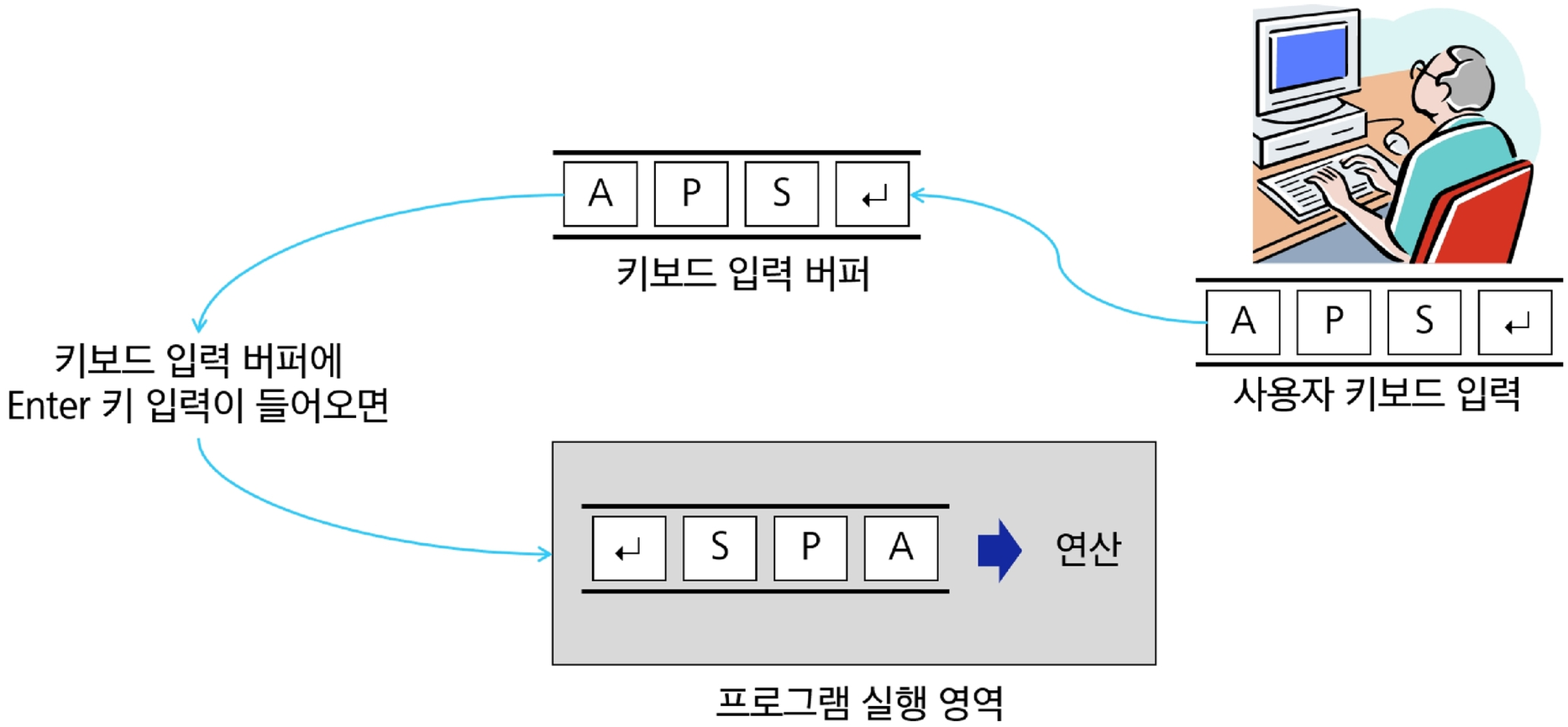
버퍼를 채우는 것을 버퍼링이라고 합니다.

버퍼의 자료 구조

- 버퍼는 일반적으로 입출력 및 네트워크와 관련된 기능에서 이용됨
- 순서대로 입력/출력/전달되어야 하므로 FIFO 방식의 자료구조인 큐가 활용됨

버퍼의 예시 : 키보드 버퍼

- 키보드 버퍼는 다음과 같이 실행됩니다



<그림17_SW문제해결 기본_Queue_버퍼의 예시 키보드 입력 >

큐를 이용한 마이쮸 나눠주기 시뮬레이션

1번이 줄을 선다.
1번이 한 개의 마이쮸를 받는다.

→

1번이 다시 줄을 선다.
새로 2번이 들어와 줄을 선다.

→

1번이 두 개의 마이쮸를 받는다.
1번 다시 줄을 선다.
새로 3번이 들어와 줄을 선다.

→

2번이 한 개의 마이쮸를 받는다.
2번이 다시 줄을 선다.
새로 4번이 들어와 줄을 선다.

→

1번이 세 개의 마이쮸를 받는다.
1번이 다시 줄을 선다.
새로 5번이 들어와 줄을 선다.

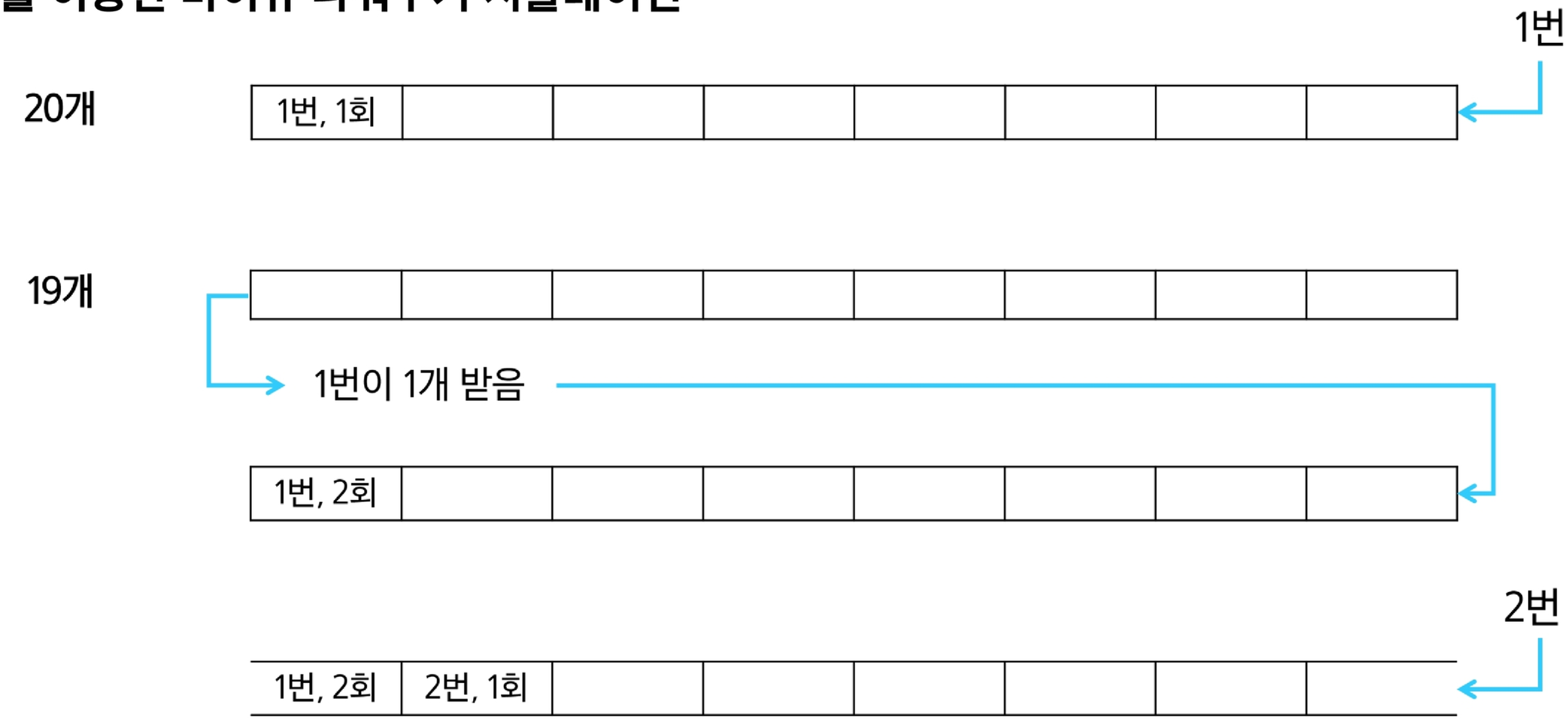
→

3번이 한 개의 마이쮸를 받는다.

…

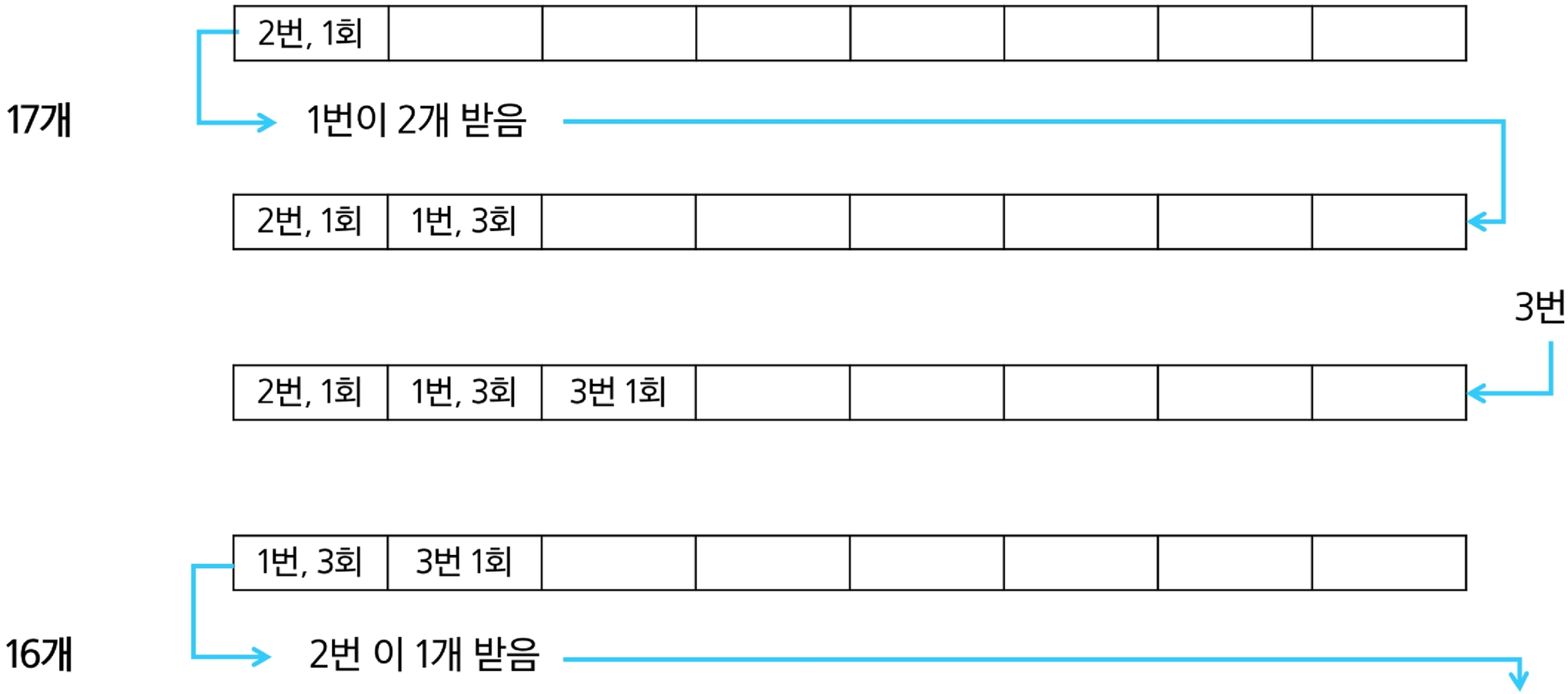
20개의 마이쮸가 있을 때 마지막 것을 누가 가져갈까?

큐를 이용한 마이쮸 나눠주기 시뮬레이션



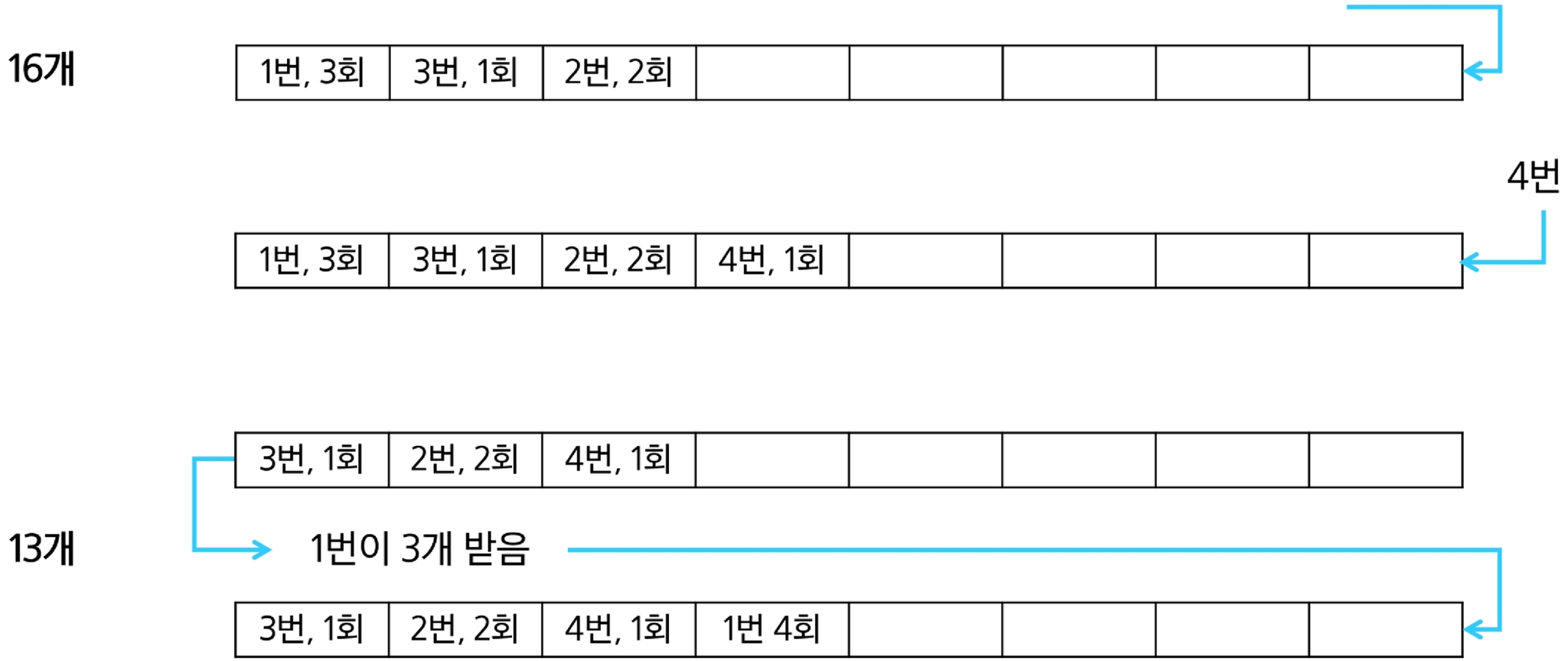
<그림18_SW문제해결 기본_Queue_마이쮸 시뮬레이션 1>

큐를 이용한 마이쮸 나눠주기 시뮬레이션



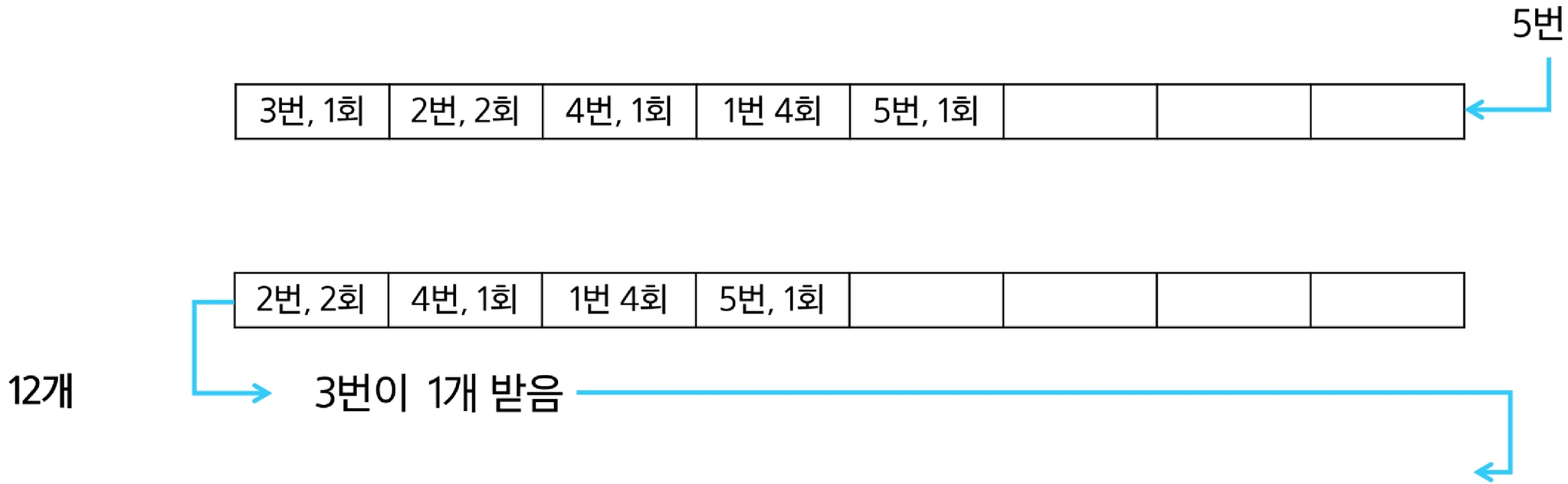
<그림19_SW문제해결 기본_Queue_마이쮸 시뮬레이션 2>

큐를 이용한 마이쮸 나눠주기 시뮬레이션



<그림20_SW문제해결 기본_Queue_마이쮸 시뮬레이션 3 >

큐를 이용한 마이쮸 나눠주기 시뮬레이션



<그림21_SW문제해결 기본_Queue_마이쮸 시뮬레이션 4>

...

마지막 마이쮸는 누가 가져갈까?

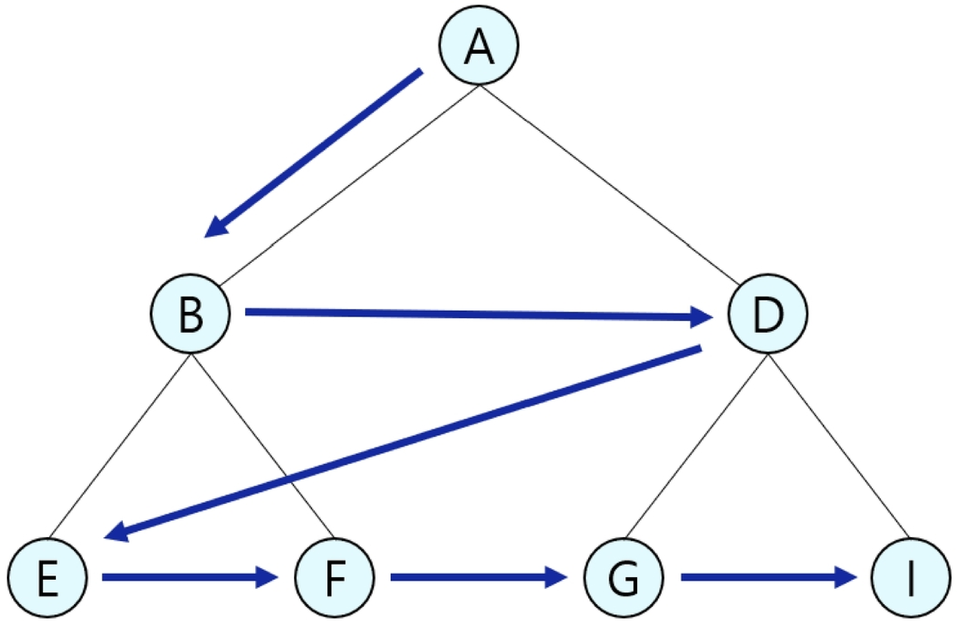
큐를 이용한 마이쮸 나눠주기 시뮬레이션

- 마이쮸 시뮬레이션 구현하기
- 엔터를 칠 때마다 다음 정보를 화면에 출력
  - 큐에 있는 사람 수
  - 현재 1인당 나눠주는 사탕의 수
  - 현재까지 나눠준 사탕의 수

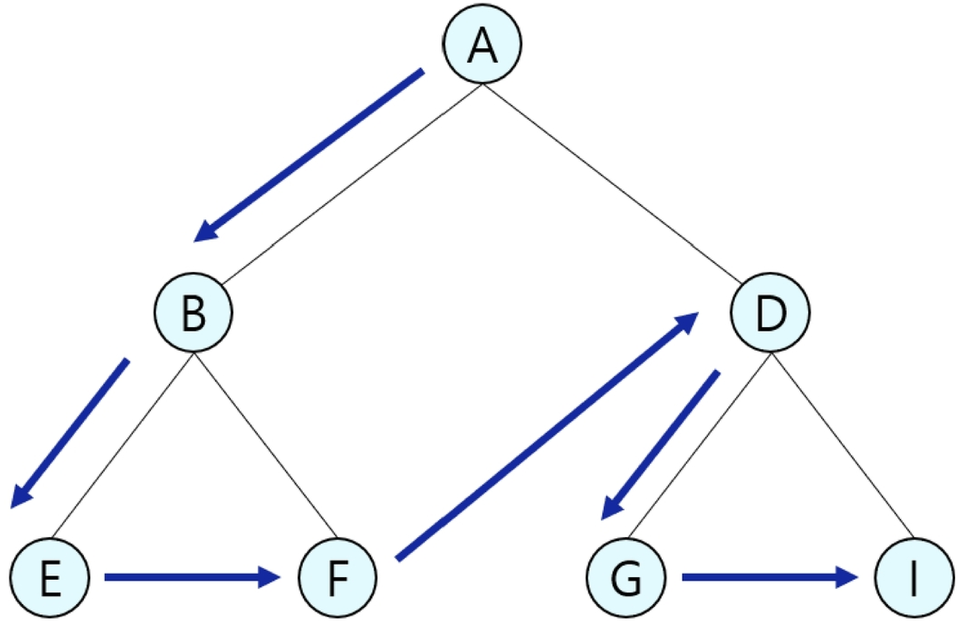
BFS

그래프를 탐색하는 방법

너비 우선 탐색(Breadth First Search, BFS)



깊이 우선 탐색(Depth First Search, DFS)



<그림22_SW문제해결 기본_Queue_BFS와 DFS>

너비우선탐색(BFS)의 개념

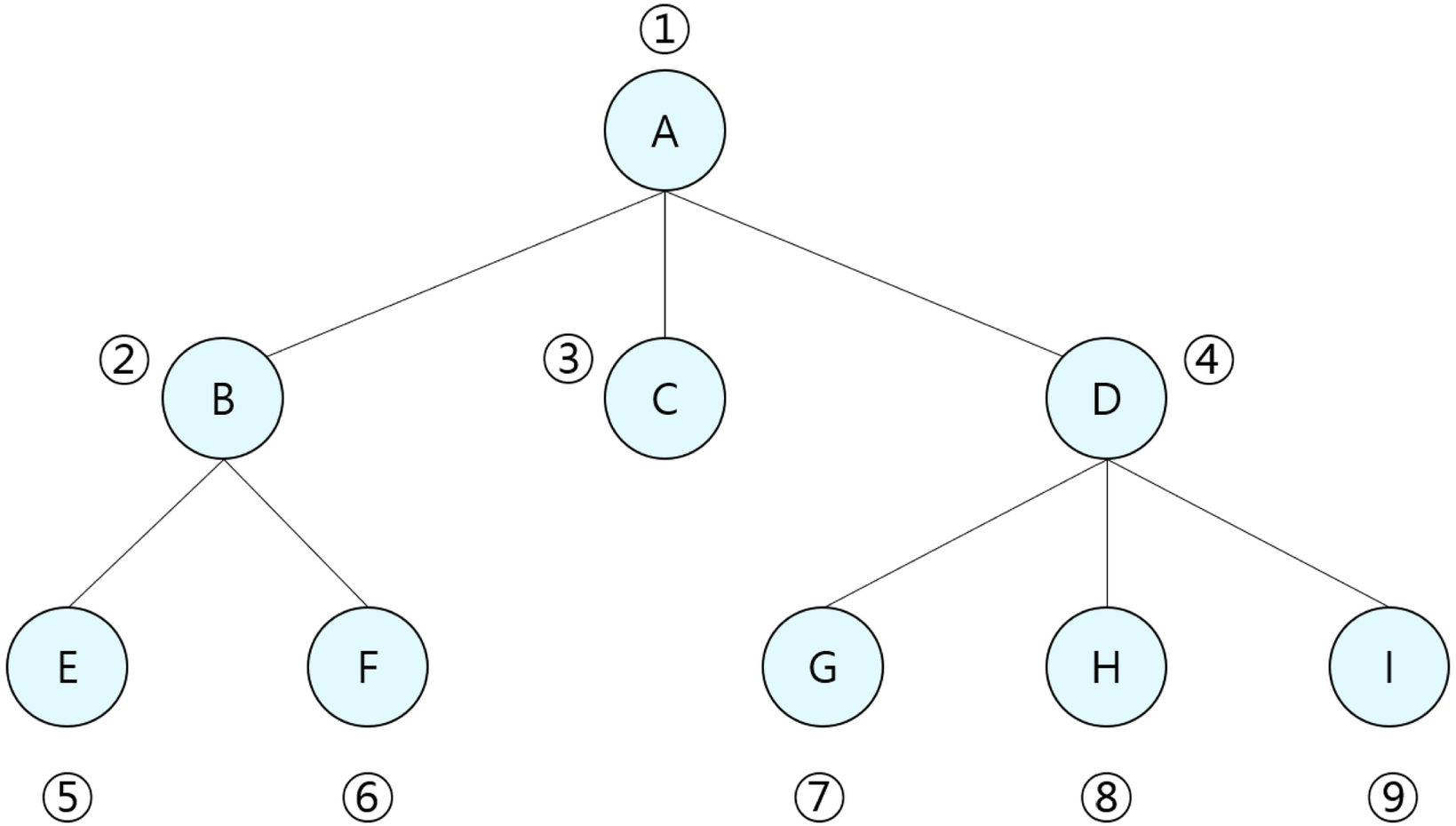
너비 우선 탐색

BFS, Breadth First Search

탐색 시작 정점에 인접한 정점들을 모두 차례로 방문한 후에,
방문했던 정점을 시작점으로 하여 다시 인접한 정점들을 차례로 방문하는 방식

인접한 정점들에 대해 탐색을 한 후, 차례로 다시 너비우선탐색을 진행해야 하므로,
선입선출 형태의 자료구조인 큐를 활용합니다.

BFS의 탐색 순서



<그림23_SW문제해결 기본_Queue_BFS>

BFS 알고리즘

- 입력 파라미터 : 그래프 G와 탐색 시작점 v

```
def bfs(G, v) : # 그래프 G, 탐색 시작점 v
    visited = [0]*(n+1)          # n : 정점의 개수
    queue = []                   # 큐 생성
    queue.append(v)              # 시작점 v를 큐에 삽입
    while queue :                        # 큐가 비어 있지 않은 경우
        t = queue.pop(0)                 # 큐의 첫 번째 원소 반환
        if not visited[t] :              # 방문하지 않은 곳이면
            visited[t] = True                    # 방문한 것으로 표시
            visit(t)                             # 정점 t에서 할 일
            for i in G[t] :                      # t와 연결된 모든 정점에 대해
                if not visited[i] :                      # 방문하지 않은 곳이면
                    queue.append(i)                              # 큐에 넣기
```

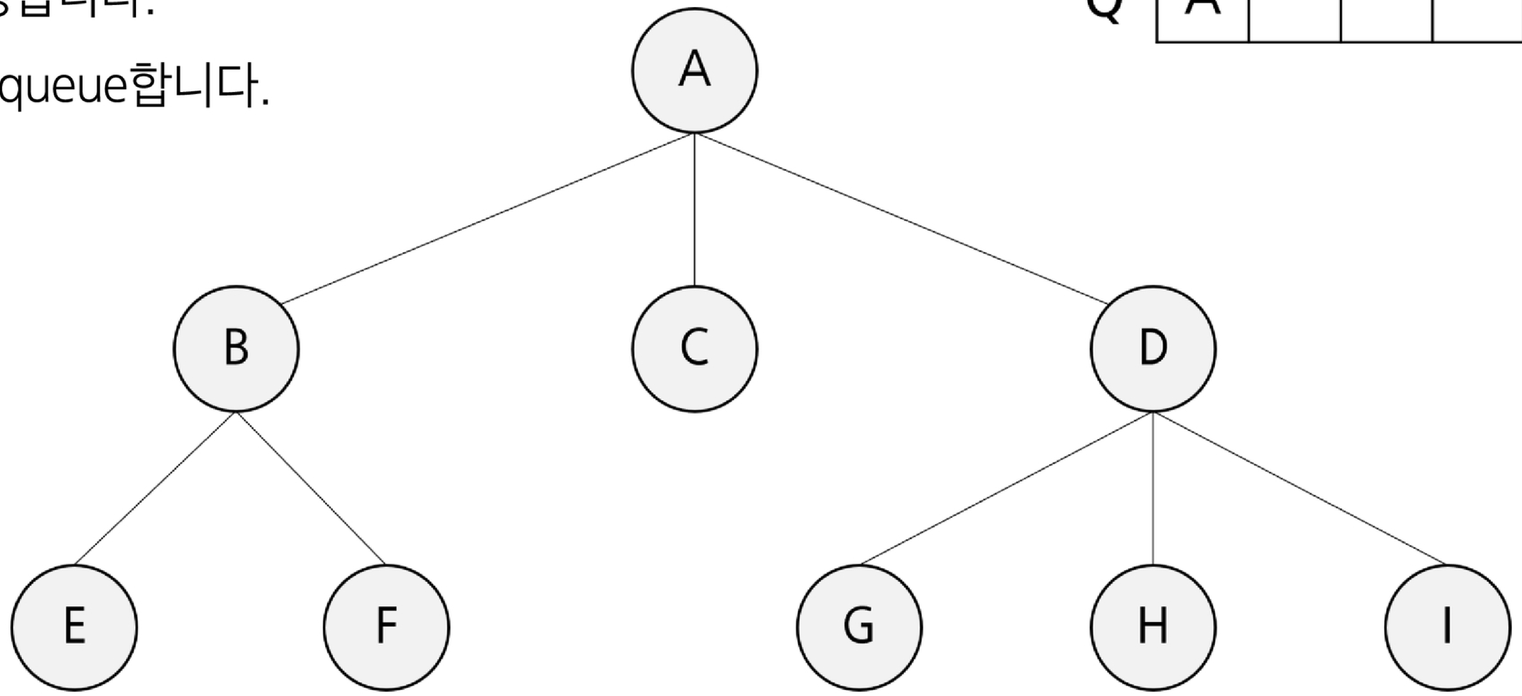
BFS 예제 1

- 초기 상태

  - visited 배열을 만들고 초기화합니다.
  - Q(큐)를 생성합니다.
  - 시작점을 enqueue합니다.

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	F	F	F	F	F	F	F	F	F

Q	A								



<그림24_SW문제해결 기본_Queue_BFS 초기 상태>

BFS 예제 1

- A점부터 시작

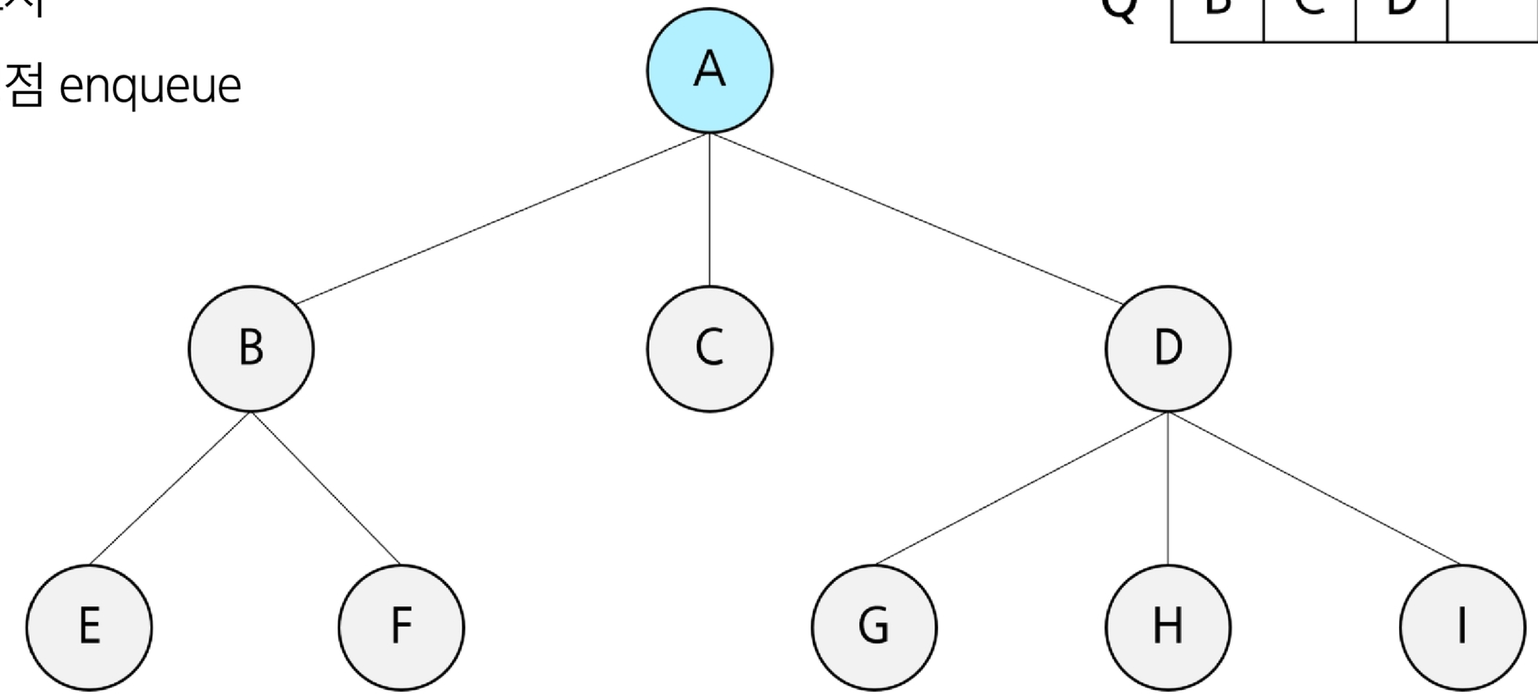
- dequeue : A

- A에 방문 표시

- A의 인접 정점 enqueue

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	T	F	F	F	F	F	F	F	F

Q	B	C	D						



<그림25_SW문제해결 기본_Queue_BFS 시작>

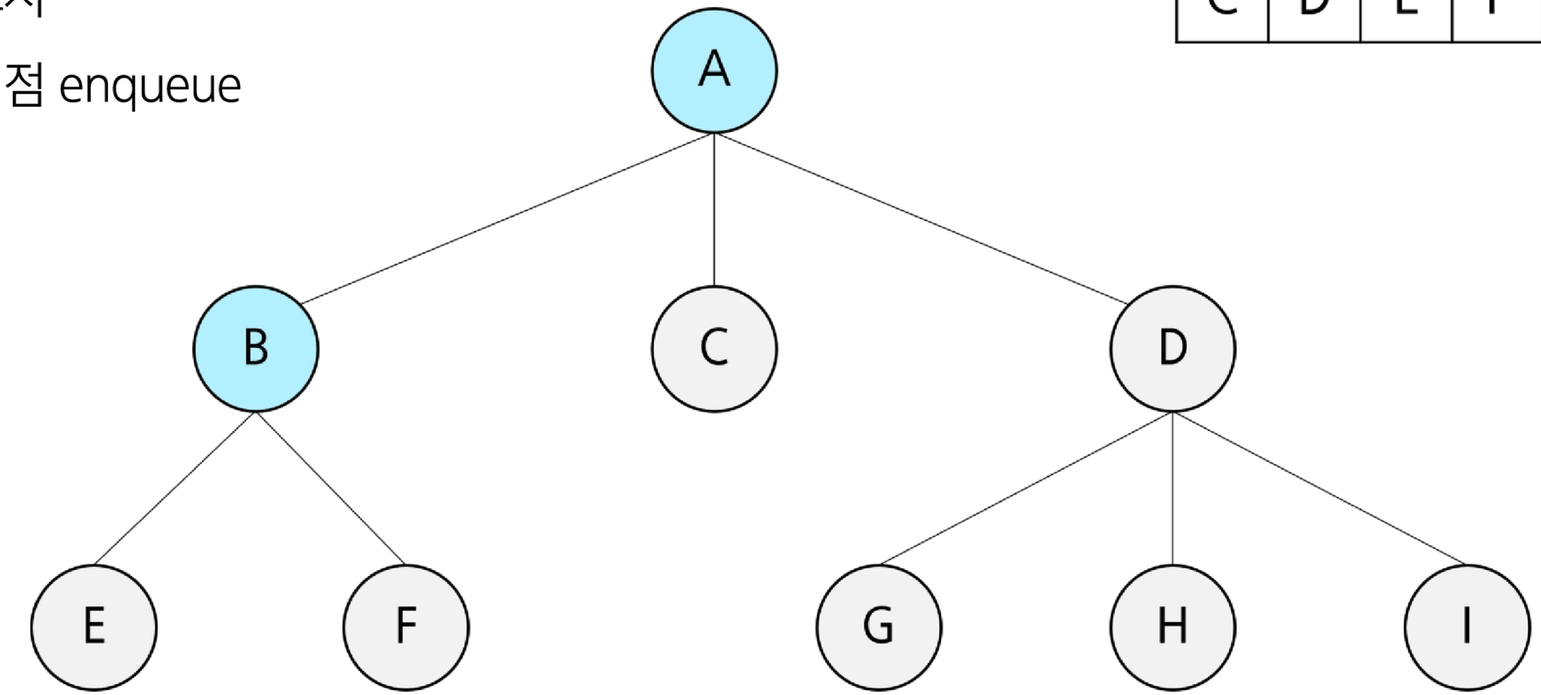
BFS 예제 1

- 탐색 진행

- dequeue : B
- B에 방문 표시
- B의 인접 정점 enqueue

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	T	T	F	F	F	F	F	F	F

C	D	E	F					



<그림26_SW문제해결 기본_Queue_BFS B 탐색>

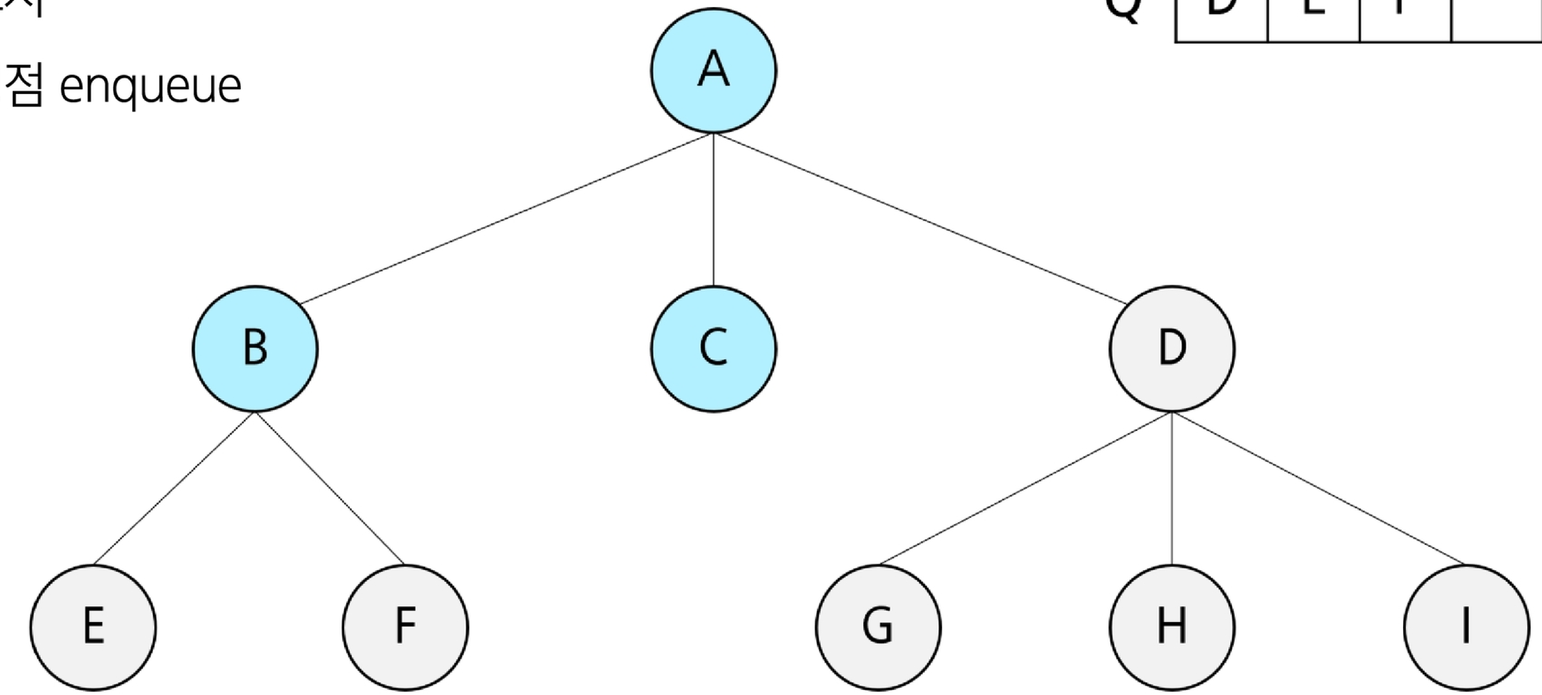
BFS 예제 1

- 탐색 진행

- dequeue : C
- C에 방문 표시
- C의 인접 정점 enqueue

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	T	T	T	F	F	F	F	F	F

Q	D	E	F						



<그림27_SW문제해결 기본_Queue_BFS C 탐색>

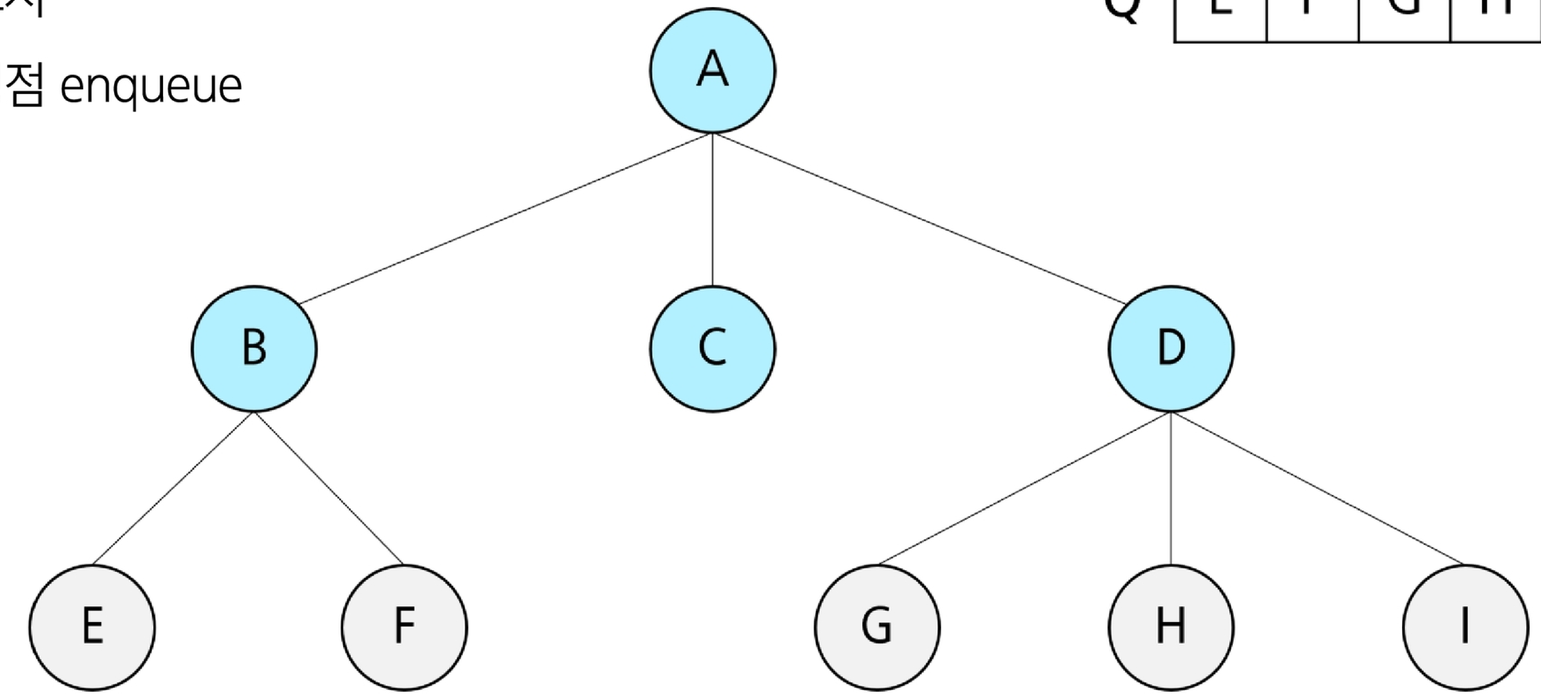
BFS 예제 1

- 탐색 진행

  - dequeue : D
  - D에 방문 표시
  - D의 인접 정점 enqueue

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	T	T	T	T	F	F	F	F	F

Q	E	F	G	H	I				



<그림28_SW문제해결 기본_Queue_BFS D 탐색>

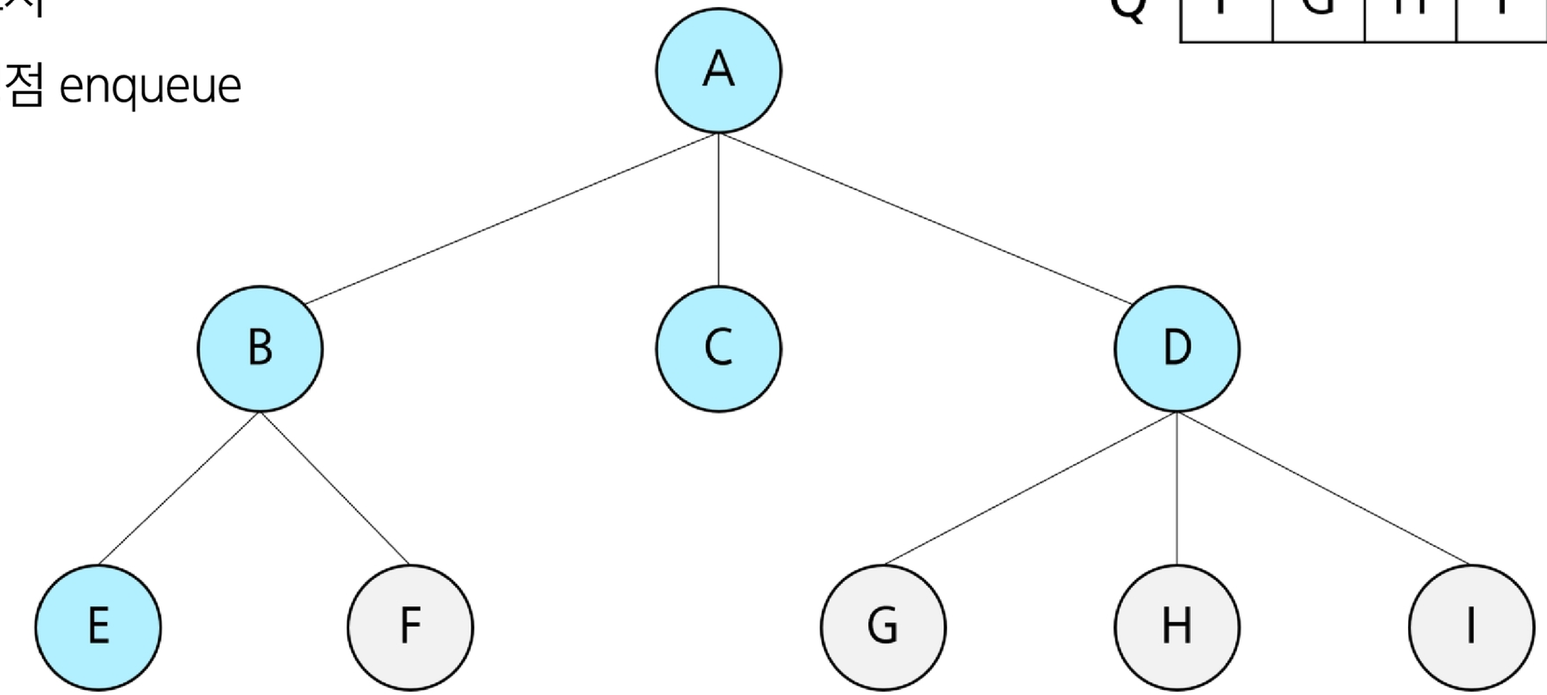
BFS 예제 1

- 탐색 진행

- dequeue : E
- E에 방문 표시
- E의 인접 정점 enqueue

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	T	T	T	T	T	F	F	F	F

Q	F	G	H	I					



<그림29_SW문제해결 기본_Queue_BFS E 탐색>

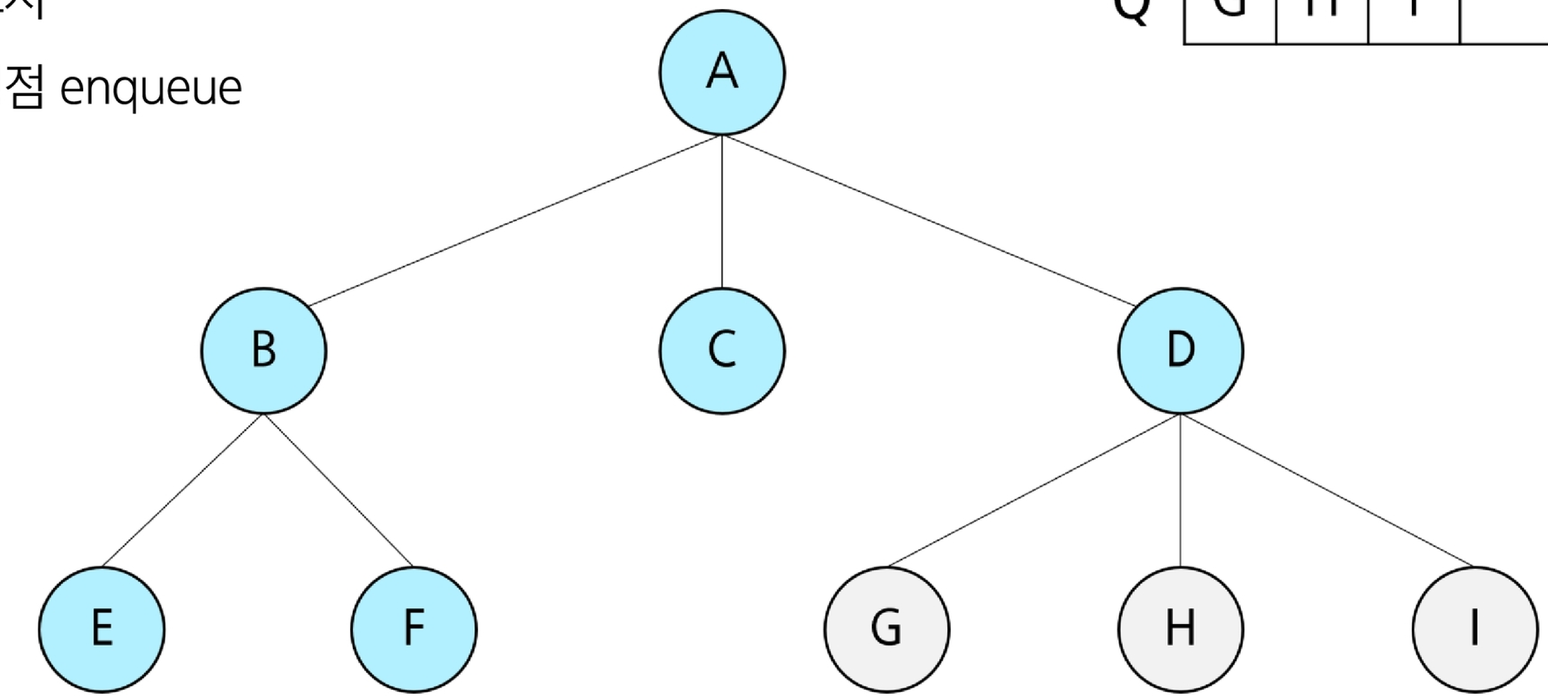
BFS 예제 1

- 탐색 진행

- dequeue : F
- F에 방문 표시
- F의 인접 정점 enqueue

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	T	T	T	T	T	T	F	F	F

Q	G	H	I						



<그림30_SW문제해결 기본_Queue_BFS F 탐색>

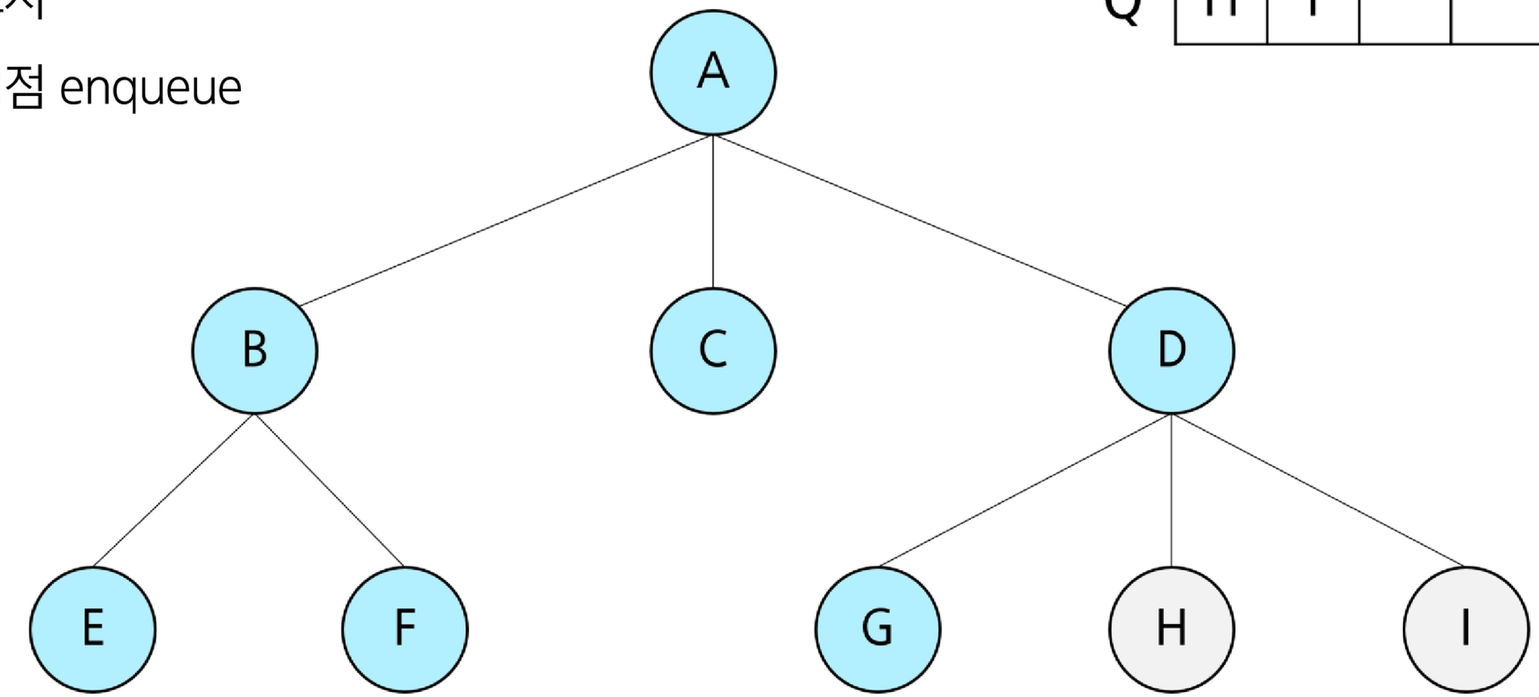
BFS 예제 1

- 탐색 진행

- dequeue : G
- G에 방문 표시
- G의 인접 정점 enqueue

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	T	T	T	T	T	T	T	F	F

Q	H	I							



<그림31_SW문제해결 기본_Queue_BFS G 탐색>

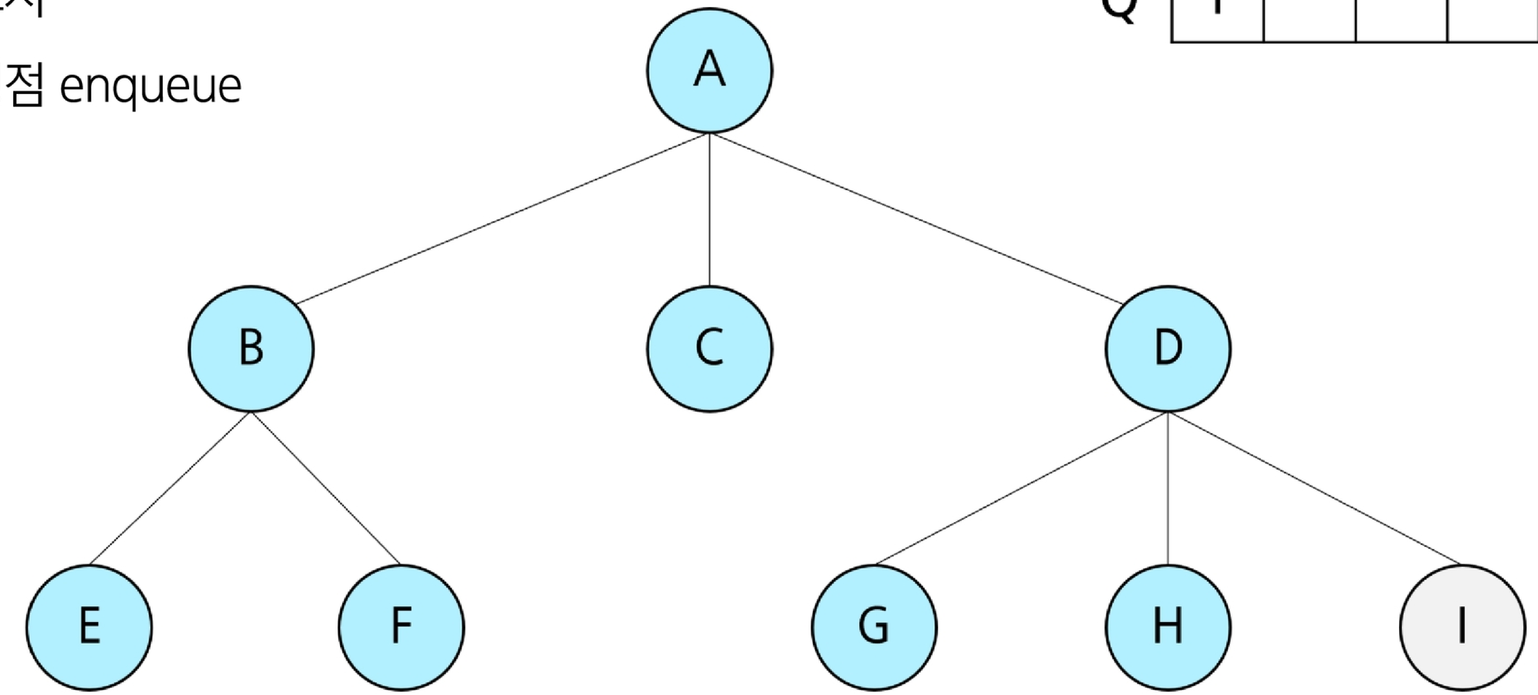
BFS 예제 1

- 탐색 진행

- dequeue : H
- H에 방문 표시
- H의 인접 정점 enqueue

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	T	T	T	T	T	T	T	T	F

Q	I								



<그림32_SW문제해결 기본_Queue_BFS H 탐색>

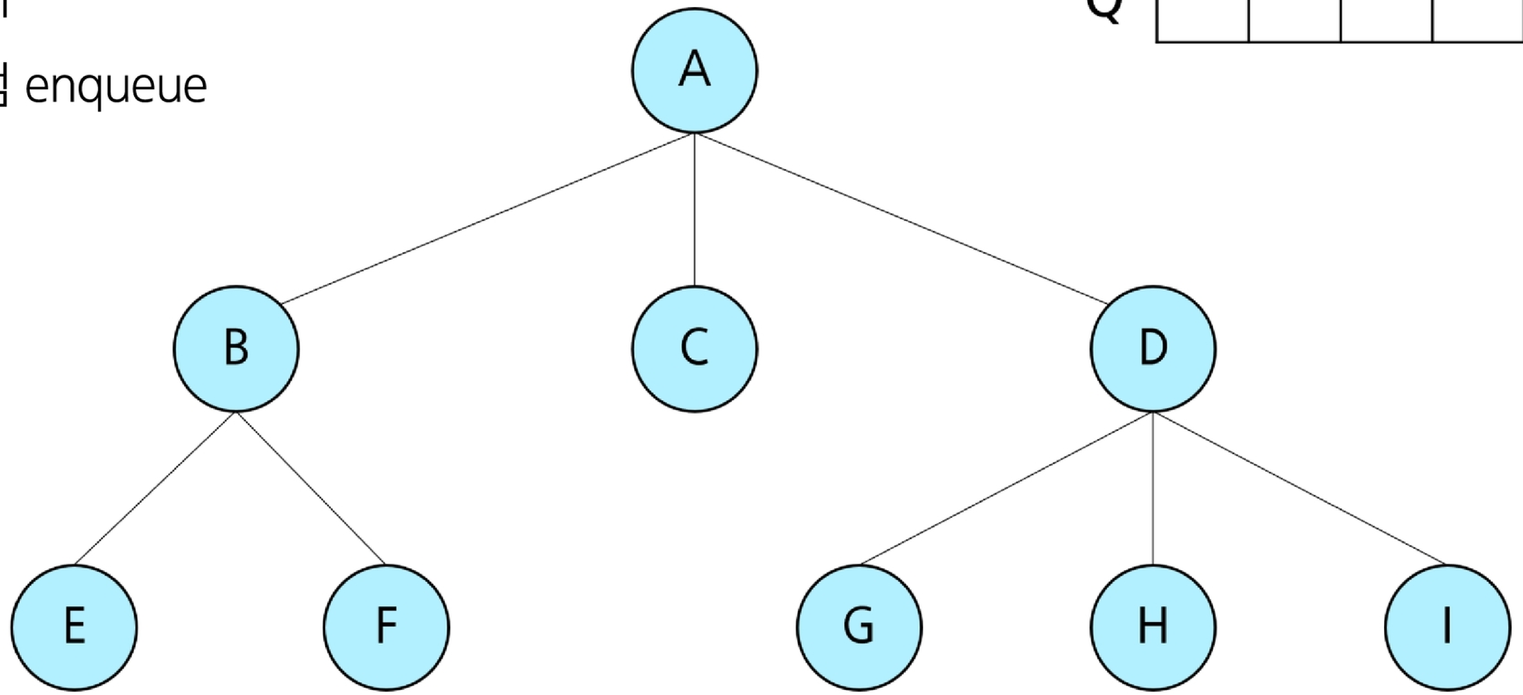
BFS 예제 1

- 탐색 진행

- dequeue : I
- I에 방문 표시
- I의 인접 정점 enqueue

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	T	T	T	T	T	T	T	T	T

Q									



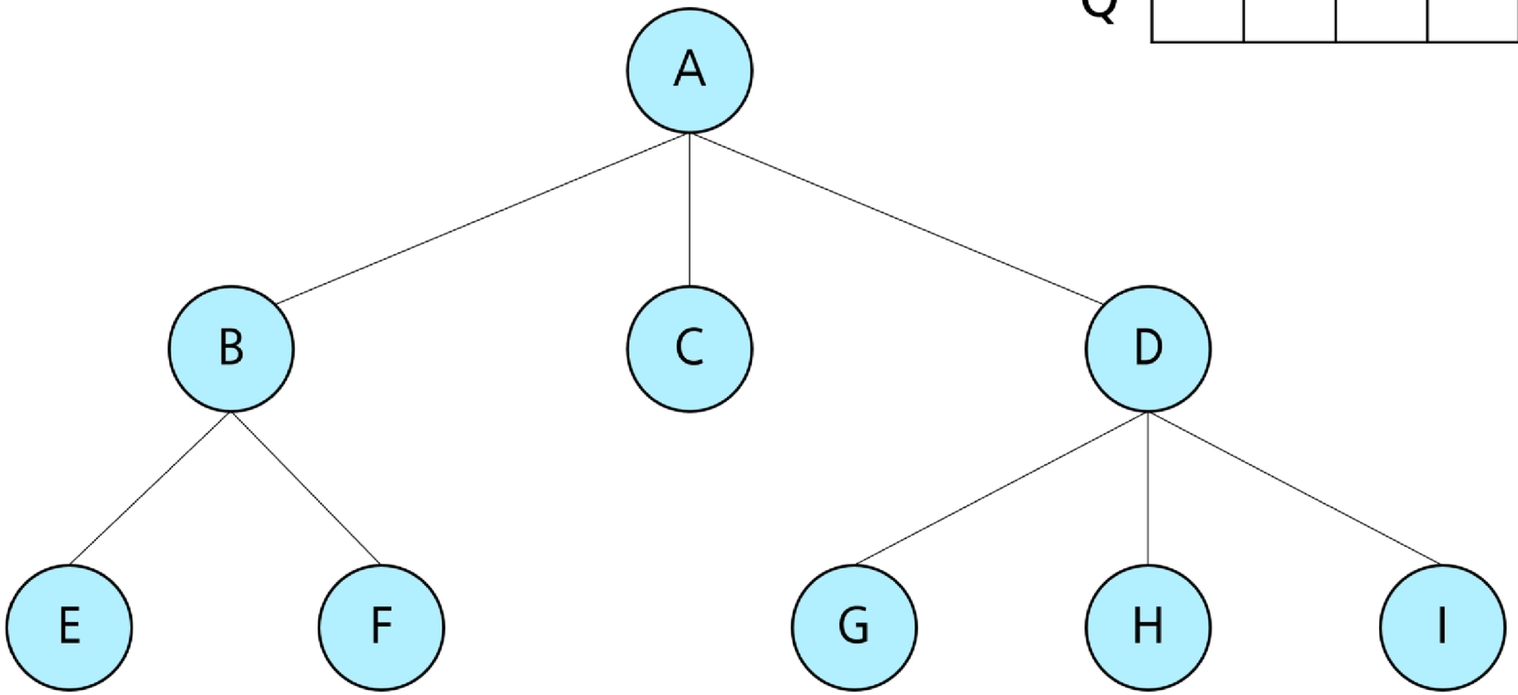
<그림33_SW문제해결 기본_Queue_BFS I 탐색>

BFS 예제 1

- Q가 비었으므로 탐색 종료

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited	T	T	T	T	T	T	T	T	T

Q									



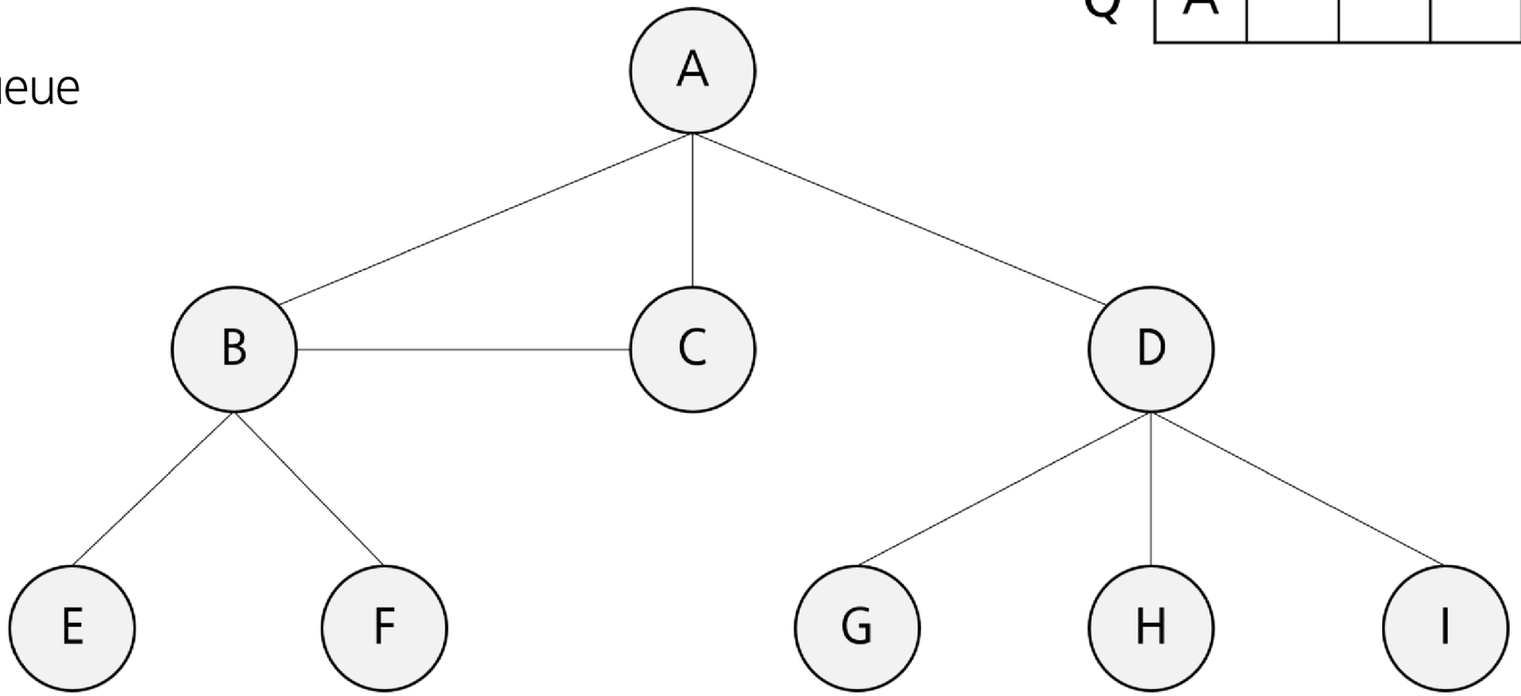
<그림34_SW문제해결 기본_Queue_BFS 탐색 종료>

BFS 예제 2

- 초기 상태
  - visited 배열 초기화
  - Q 생성
  - 시작점 enqueue

	A	B	C	D	E	F	G	H	I
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
visited									

Q	A								



<그림35_SW문제해결 기본_Queue_BFS 예제 2>

BFS 예제 2

- 입력 파라미터 : 그래프 G와 탐색 시작점 v

```
def bfs(G, v, n) : # 그래프 G, 탐색 시작점 v
    visited = [0]*(n+1)          # n : 정점의 개수
    queue = []                   # 큐 생성
    queue.append(v)              # 시작점 v를 큐에 삽입
    visited[v] = 1
    while queue :                          # 큐가 비어 있지 않은 경우
        t = queue.pop(0)                         # 큐의 첫 번째 원소 반환
        visit(t)
        for i in G[t] :                          # t와 연결된 모든 정점에 대해
            if not visited[i] :                  # 인큐되지 않은 곳이라면
                queue.append(i)                        # 큐에 넣기
                visited[i] = visited[t] + 1            # n으로부터 1만큼 이동
```

BFS 예제 3

- 코드 : 6_Queue/queue_ex3.py

다음은 연결되어 있는 두 개의 정점 사이의 간선을 순서대로 나열 해 놓은 것이다. 모든 정점을 너비 우선 탐색 하여 경로를 출력하시오. 시작 정점을 1로 시작하시오.

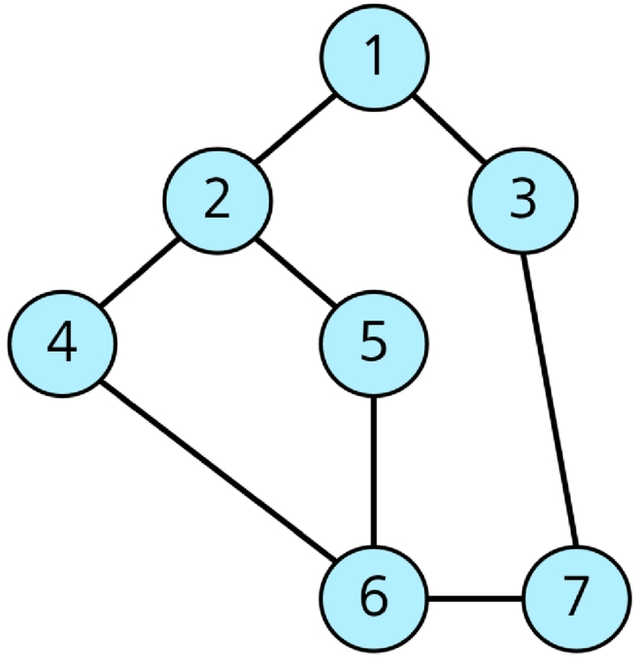
입력

7 8 # 정점 수, 간선 수

1 2 1 3 2 4 2 5 4 6 5 6 6 7 3 7

출력 결과의 예는 다음과 같다.

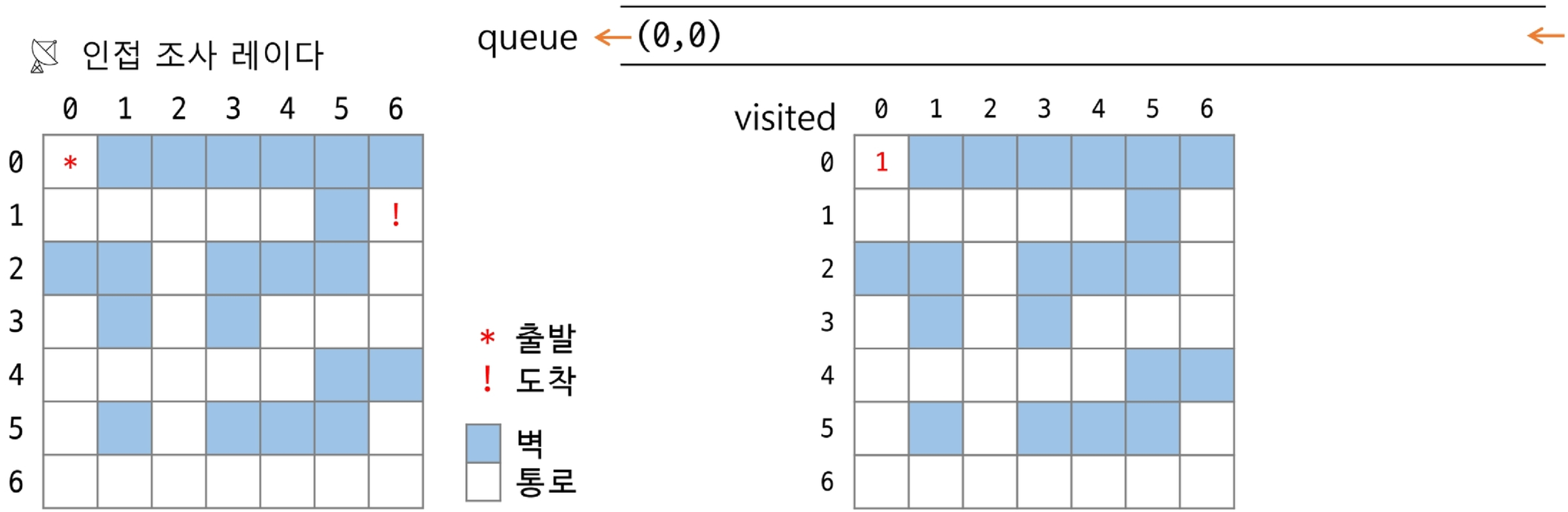
1 2 3 4 5 7 6



<그림36_SW문제해결 기본_Queue_BFS 예제 3>

BFS를 활용한 미로 탐색

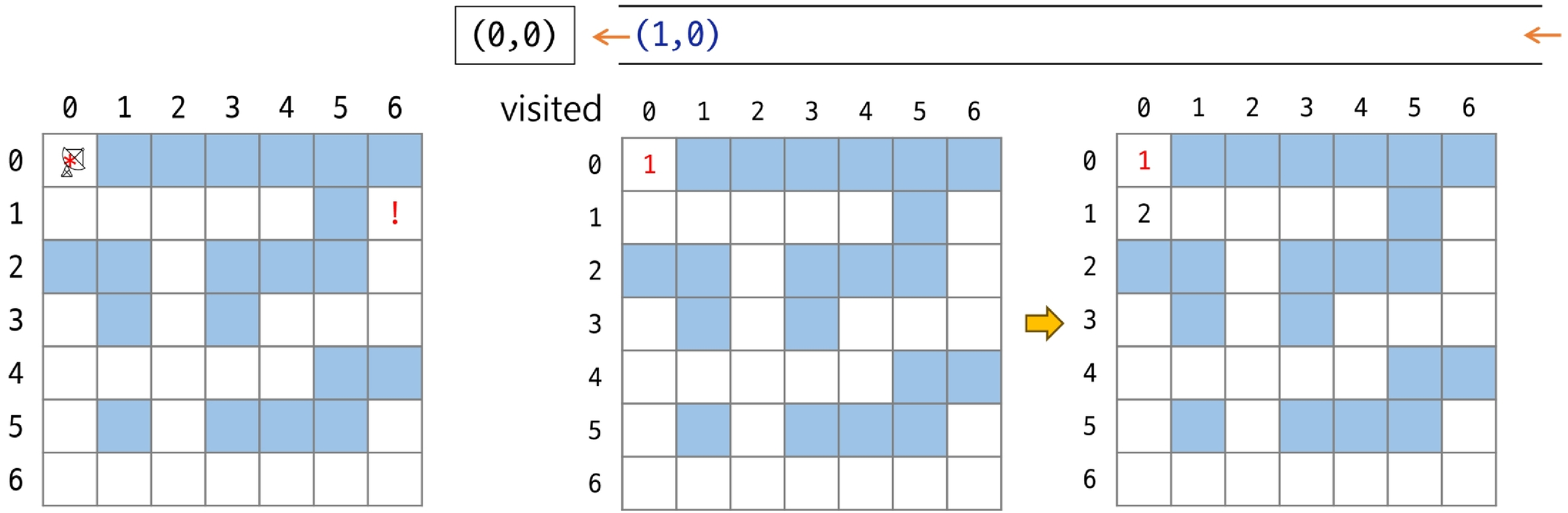
- 출발에서 도착까지 최단 거리 탐색
  - 큐 생성, 시작점을 대기 목록에 기록(enqueue)
  - visited에 표시



<그림37_SW문제해결 기본_Queue_BFS 미로 탐색 1>

BFS를 활용한 미로 탐색

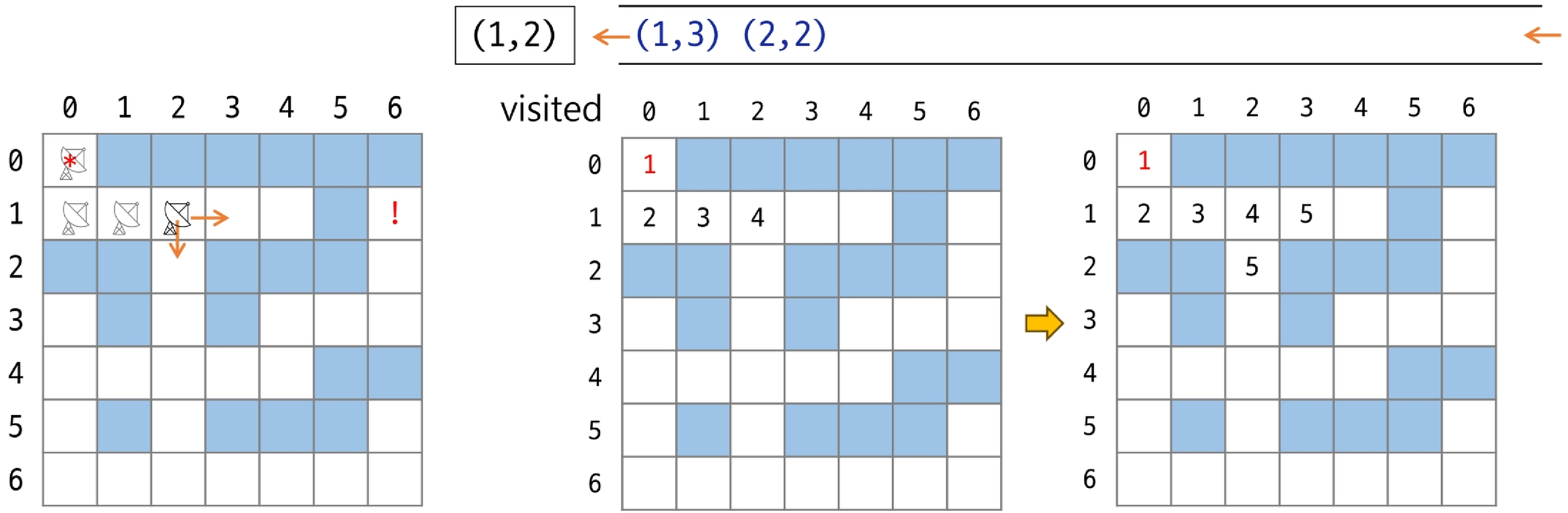
- 큐가 비워질 때까지 반복
  - 디큐한 좌표 (i,j) 탐색 (예 : 도착지면 중단)
  - (i,j)에 인접하고 아직 목록에 없으면 목록에 올리고 visited 표시



<그림38_SW문제해결 기본_Queue_BFS 미로 탐색 2>

BFS를 활용한 미로 탐색

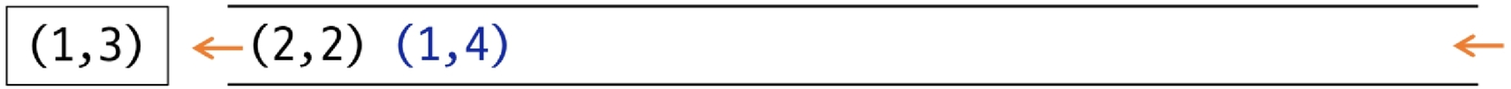
- 큐가 비워질 때까지 반복
  - 디큐한 좌표 (i,j) 탐색 (예 : 도착지면 중단)
  - (i,j)에 인접하고 아직 목록에 없으면 목록에 올리고 visited 표시

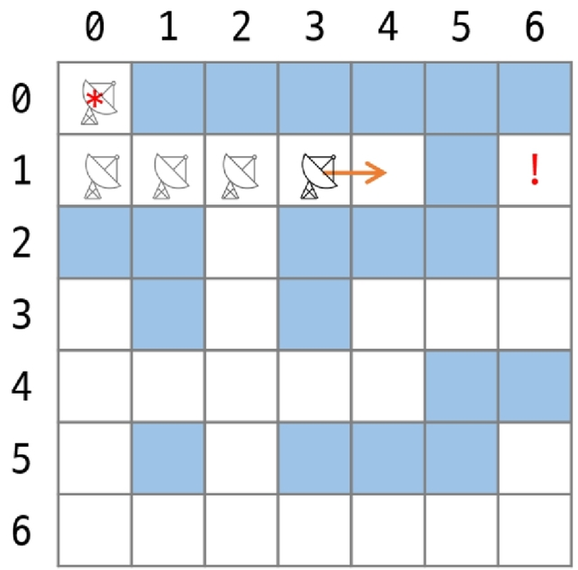


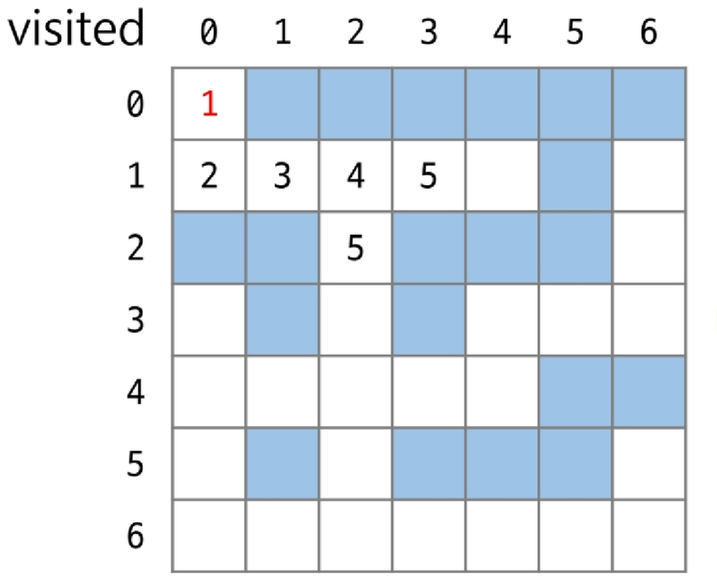
<그림39_SW문제해결 기본_Queue_BFS 미로 탐색 3>

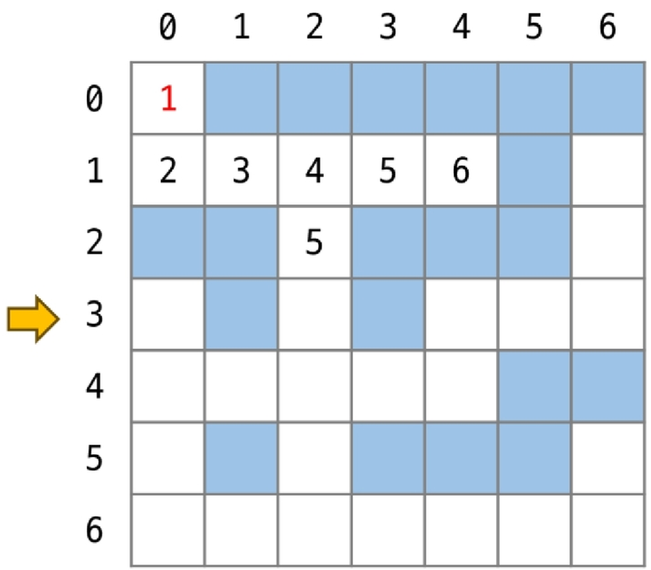
BFS를 활용한 미로 탐색

- 큐가 비워질 때까지 반복
  - 디큐한 좌표 (i,j) 탐색 (예 : 도착지면 중단)
  - (i,j)에 인접하고 아직 목록에 없으면 목록에 올리고 visited 표시







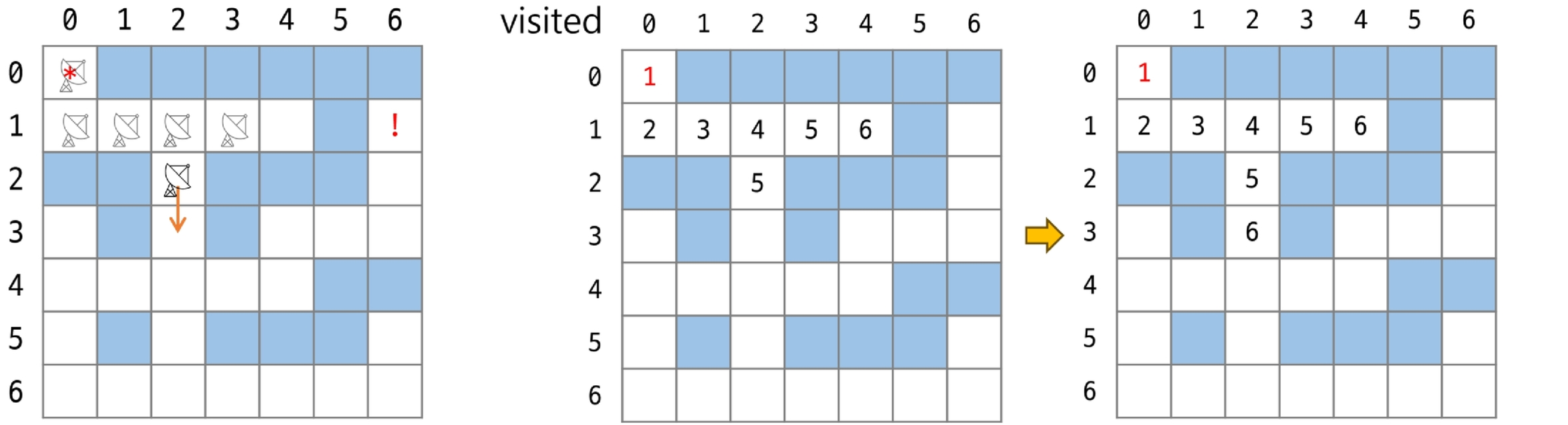


<그림40_SW문제해결 기본_Queue_BFS 미로 탐색 4>

BFS를 활용한 미로 탐색

- 큐가 비워질 때까지 반복
  - 디큐한 좌표 (i,j) 탐색 (예 : 도착지면 중단)
  - (i,j)에 인접하고 아직 목록에 없으면 목록에 올리고 visited 표시

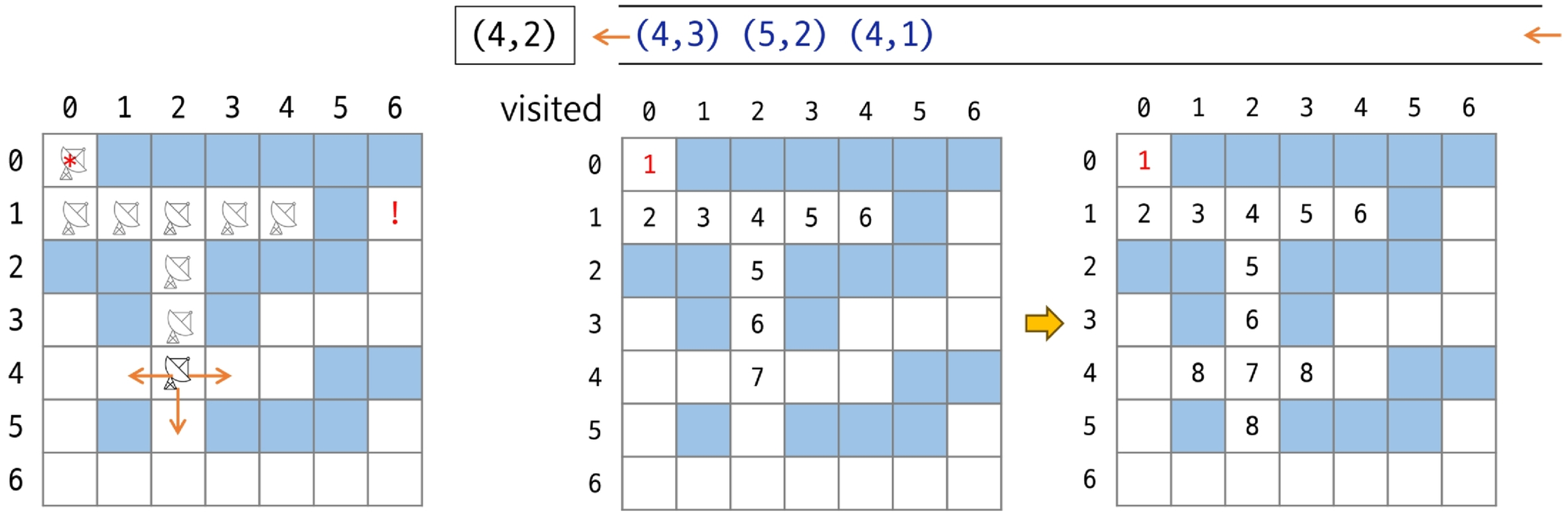
(2,2) ← (1,4) (3,2) ←



<그림41_SW문제해결 기본_Queue_BFS 미로 탐색 5>

BFS를 활용한 미로 탐색

- 큐가 비워질 때까지 반복
  - 디큐한 좌표 (i,j) 탐색 (예 : 도착지면 중단)
  - (i,j)에 인접하고 아직 목록에 없으면 목록에 올리고 visited 표시



<그림42_SW문제해결 기본_Queue_BFS 미로 탐색 6>

실습

큐

- 5097. 회전

시뮬레이션

- 5099. 피자 굽기

BFS

- 5102. 노드의 거리
- 5105. 미로의 거리

추가 연습

- 10966. 물놀이를 가자

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 질문에 답하시오. (단답형)

큐의 데이터 입출력 순서는?

2 다음 중 큐의 기본 연산이 아닌 것은?

a) enqueue
b) dequeue
c) peek
d) push

3 다음 중 큐의 종류로 볼 수 없는 것은?

a) 선형 큐
b) 우선순위 큐
c) 원형 큐
d) 그래프 큐

4 다음 중 배열로 구현한 선형 큐의 단점은?

a) 구현이 복잡하다
b) 사용한 공간을 재사용할 수 없다
c) 데이터를 뒤에서 꺼낸다
d) 메모리를 무제한으로 사용한다

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 다음 질문에 답하시오. (단답형)

선형 큐의 공간 낭비를 해결하기 위해 고안된 큐는?

6 다음 중 연결 리스트로 구현할 수 있는 큐는?

a) 원형 큐
b) 연결 큐
c) 우선순위 큐
d) 모두 해당

7 다음 중 큐를 활용한 예시로 적절하지 않은 것은?

a) 고객 대기열
b) 명령어 실행 기록
c) 프린터 작업 대기열
d) 패킷 전송 버퍼

8 다음 중 큐가 사용되는 알고리즘은?

a) DFS
b) BFS
c) 피보나치 수열
d) 이진 탐색

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 다음 중 큐가 아닌 다른 자료구조를 사용하는 것은?

a) 너비 우선 탐색
b) 프린터 버퍼
c) 브라우저 뒤로가기
d) 패킷 처리 대기열

10 버퍼에서 큐를 사용하는 이유는 무엇인가요?

a) 가장 최근 요청을 먼저 처리하기 위해
b) 먼저 들어온 요청부터 순서대로 처리하기 위해
c) 데이터를 한꺼번에 처리하기 위해
d) 저장 공간을 최소화하기 위해

11 다음 중 큐의 동작을 직접 구현한 함수로 알맞은 것은?(복수 선택)

a)
```
def enqueue(q, x):
    q.insert(0, x)
```

b)
```
def dequeue(q):
    return q.pop()
```

c)
```
def enqueue(q, x):
    q.append(x)
```

d)
```
def dequeue(q):
    return q.pop(0)
```

확인문제

정답 및 해설

1. FIFO, 선입선출 2. d) push 3. d) 그래프 큐 4. b) 사용한 공간을 재사용할 수 없다 5. 원형 큐 6. d) 모두 해당 7. b) 명령어 실행 기록 8. b) BFS 9. c) 브라우저 뒤로가기 10. b) 먼저 들어온 요청부터 순서대로 처리하기 위해 11. c)와 d)

1. 큐는 First-In-First-Out(선입선출) 구조로, 먼저 들어온 데이터가 먼저 나갑니다.

2. push는 스택 연산이고, 큐는 enqueue, dequeue, peek 등을 사용합니다.

3. 그래프 큐는 존재하지 않으며, 선형 큐, 원형 큐, 우선순위 큐는 실제 큐의 다양한 형태입니다.

4. 선형 큐는 앞부분이 비어도 다시 사용할 수 없기 때문에 낭비가 생깁니다.

5. 원형 큐는 배열을 순환시켜 앞쪽 공간을 재사용할 수 있게 합니다.

6. 연결 리스트로 큐의 다양한 형태를 구현할 수 있습니다.

7. 명령어 실행 기록은 후입선출 방식의 스택이 적절합니다.

8. BFS는 너비 우선 탐색으로, 큐를 사용해 가장 가까운 노드부터 탐색합니다.

9. 브라우저 뒤로가기는 후입선출 방식의 스택 구조를 사용합니다.

10. 버퍼는 순차적인 데이터 흐름을 관리하므로 큐(FIFO) 구조가 적합합니다.

11. 큐는 뒤에 추가하고(append), 앞에서 제거(pop(0))하는 방식으로 동작합니다.

활동정리

핵심키워드

개념	설명	예시
큐(Queue)	먼저 들어온 데이터가 먼저 나가는 선형 자료구조	은행 대기 줄, 프린터 대기열
선입선출(FIFO)	먼저 들어온 데이터가 먼저 나가는 자료 처리 방식	1 → 2 → 3 순서로 넣으면 1 → 2 → 3 순서로 꺼냄
덱(Deque)	양쪽 끝에서 빠르게 추가와 삭제를 할 수 있는 큐	웹 브라우저 방문 기록(앞/뒤 이동)
버퍼(Buffer)	일시적으로 데이터를 보관하는 메모리의 영역	영상 스트리밍 버퍼, 키보드 입력
너비우선탐색(BFS)	시작점의 인접한 정점들을 모두 차례로 방문한 후, 방문했던 정점을 시작점으로 하여 다시 인접한 정점들을 차례로 방문하는 방식	최단 경로 탐색, 미로 찾기

<표3_SW문제해결 기본_Queue_핵심키워드>

활동정리

오늘 공부한 내용 요약 및 정리

- **큐(Queue)란?**
  - 먼저 들어온 데이터가 먼저 나가는 선입선출(FIFO) 자료구조
  - 자료의 삽입(enqueue), 삭제(dequeue), 공백 상태 확인(is_empty), 포화 상태 확인(is_full) 등의 기본 연산 필요
  - 대표적인 큐의 종류로 선형 큐, 원형 큐, 연결 큐, 우선순위 큐가 있음

- **선입선출(FIFO, First In First Out) 구조**
  - 먼저 들어온 데이터가 먼저 나가는 자료 처리 방식

활동정리

오늘 공부한 내용 요약 및 정리

- **버퍼(Buffer)**
  - 입력과 출력 속도가 다른 장치 간에 데이터를 임시로 저장해주는 공간
  - 데이터를 모아 두었다가 순서대로 처리되도록 도와주는 중간 완충 역할
  - 큐를 사용하여 먼저 들어온 데이터부터 차례대로 처리함 (FIFO)

- **너비 우선 탐색(BFS)**
  - 그래프 탐색 기법으로, 현재 정점에 인접한 정점들을 먼저 탐색
  - 큐를 사용하여 탐색 순서를 유지하며, 가까운 노드부터 차례대로 방문
  - 미로 최단 거리, 레벨 탐색, 최단 경로 문제 등에 자주 사용됨

활동정리

번호표를 뽑은 순서대로 부르는 은행 창구!

은행에서는 고객이 번호표를 뽑고 기다립니다. 번호표는 먼저 온 사람부터 부르기 때문에 아래와 같이 번호표를 뽑았다고 가정해 봅시다.

1번 고객 → 2번 고객 → 3번 고객

이후 창구 직원이 두 명의 고객을 호출했다면, 호출된 고객 번호는 어떤 순서일까요?

활동정리

번호표를 뽑은 순서대로 부르는 은행 창구!

은행에서는 고객이 번호표를 뽑고 기다립니다. 번호표는 먼저 온 사람부터 부르기 때문에 아래와 같이 번호표를 뽑았다고 가정해 봅시다.

1번 고객 → 2번 고객 → 3번 고객

이후 창구 직원이 두 명의 고객을 호출했다면, 호출된 고객 번호는 어떤 순서일까요?

정답 1, 2

```
queue = [1,2,3]
print(queue.pop(0)) # 1
print(queue.pop(0)) # 2
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다